

会议室预约与时间冲突检查系统

软件概要设计以及详细设计

专 业:	软件工程
方 向:	软件开发与测试
项目名称:	会议室预约与时间冲突检查系统
队伍名称	尖草坪游击队
项目成员:	李宗熹、翟浩森、张陈浩、闫佳馥、霍汶洊
辅导教师:	张淑娟

2026 年 9 月

目录

一、本人在该设计中具体应完成的工作（李宗熹）	1
1.1 功能职责	1
1.2 跨域边界与约束	2
二、课程设计开始以来所做的具体工作和取得的进展	2
2.1 交付范围总览	3
2.2 系统登录与预约看板入口	3
2.3 预约创建与并发控制	4
2.4 半开区间时间冲突检测	6
2.5 预约状态机与生命周期	6
2.6 预约看板、详情与我的预约	7
2.7 关键接口与测试验证	9
2.8 提交记录	10
三、目前存在问题，下一步的主要设计任务，具体设想与安排	10
一、本人在该设计中具体应完成的工作（翟浩森）	11
1.1 职责范围概述	12
1.2 与其他成员的协作边界	13
二、课程设计开始以来所做的具体工作与取得的进展	13
2.1 开发过程与提交记录	13
2.2 数据库结构设计	14
2.3 后端服务实现	14
2.4 前端页面实现与接口联调	15
2.5 系统运行效果与截图证明	15
2.6 报修工单全流程演示	17

2.7 维护计划登记	19
2.8 测试与验证工作	20
2.9 阶段性进展小结	20
三、目前存在问题、下一步主要设计任务、具体设想与安排	21
3.1 目前存在的问题	21
3.2 下一步主要设计任务与安排	21
3.3 具体技术设想	21
3.4 后续学习与答辩准备	22
一、本人在该设计中具体应完成的工作（闫佳馥）	23
1.1 功能职责	23
1.2 跨域边界与约束	23
二、课程设计开始以来所做的具体工作和取得的进展	23
2.1 交付范围总览	24
2.2 数据模型与数据库迁移	24
2.3 后端核心服务实现	25
2.4 No-Show 缺席自动判定	26
2.5 业务内通知与个人通知中心	27
2.6 前端闭环与个人工作台	28
2.7 测试与验证	30
2.8 提交记录	30
三、目前存在问题，下一步的主要设计任务，具体设想与安排	32
一 本人在该设计中具体应完成的工作（霍汶炫）	34
工作边界与验收标准	35
二 课程设计开始以来所做的具体工作和取得的进展	36

2.1 需求梳理与分支管理	36
2.2 后端审批闭环与跨域协作	37
2.3 管理后台审批页面	38
2.4 操作审计与安全导出	38
2.5 运营分析与创意功能	39
2.6 数据库设计与查询优化	40
2.7 自动化测试与阶段质量	41
2.8 当前完成度概述	41
三 目前存在的问题和下一步主要设计任务	42
3.1 目前存在的问题	42
3.2 下一步的主要设计任务	42
3.3 具体设计设想	44
3.4 下一阶段验收方式	44
四 中期检查结论	45
附录 阶段成果清单	45
一、本人在该设计中具体应完成的工作（张陈浩）	46
1.1 功能职责	46
1.2 跨域边界与约束	46
二、课程设计开始以来所做的具体工作和取得的进展	47
2.1 交付范围总览	47
2.2 安全认证与登录入口	47
2.3 数据模型与数据库迁移	48
2.4 用户与信用管理工作台	48
2.5 信用分调整与违规记录	50

2.6 预约限制（黑名单）管理	51
2.7 部门管理	52
2.8 对外预约资格判定服务	53
2.9 关键接口与测试验证	54
2.10 提交记录	55
三、目前存在问题，下一步的主要设计任务，具体设想与安排	56

一、本人在该设计中具体应完成的工作（李宗熹）

项目名称	智会会议室预约与时间冲突检查系统
负责模块	负责「会议室预约与时间冲突检查系统」中的预约调度与冲突控制核心域（reservation domain）
学生姓名	李宗熹
学号	2413042129
开发分支	Main
报告日期	2026 年 9 月 17 日

在本次课程设计中，本人担任组长，并负责「会议室预约与时间冲突检查系统」中的预约调度与冲突控制核心域（reservation domain）。reservation 域是系统最核心的业务域之一：它接收用户预约请求，在会议室可用性、开放时间、容量、跨域资格与并发安全的多重约束下，保障同一会议室在同一时段不会被重复预约，并提供看板查询、详情查看、改期、取消以及生命周期状态转换等能力。

1.1 功能职责

- 预约创建：接收会议主题、会议室、日期、开始-结束时间、参会人数、备注，完成资格/容量/开放时间/时间冲突的全链路校验；
- 预约改期：仅允许预约本人修改，新时段需重新走完整规则链（会议室行锁 + 冲突检测），且只影响该预约自身；
- 预约取消：普通用户仅可取消本人处于 PENDING/CONFIRMED 且未开始的预约；管理员通过 administration 域走强制取消路径；
- 半开区间时间冲突检测：基于数据库的半开区间重叠查询与领域对象 TimeInterval 的双重校验；
- 并发控制：在创建/改期前先对会议室行加锁，结合 requestId 幂等键避免重复提交与并发误报冲突；
- 状态机与生命周期：PENDING/CONFIRMED/REJECTED/CANCELLED 状态的唯一写入入口集中为 ReservationLifecycleService；

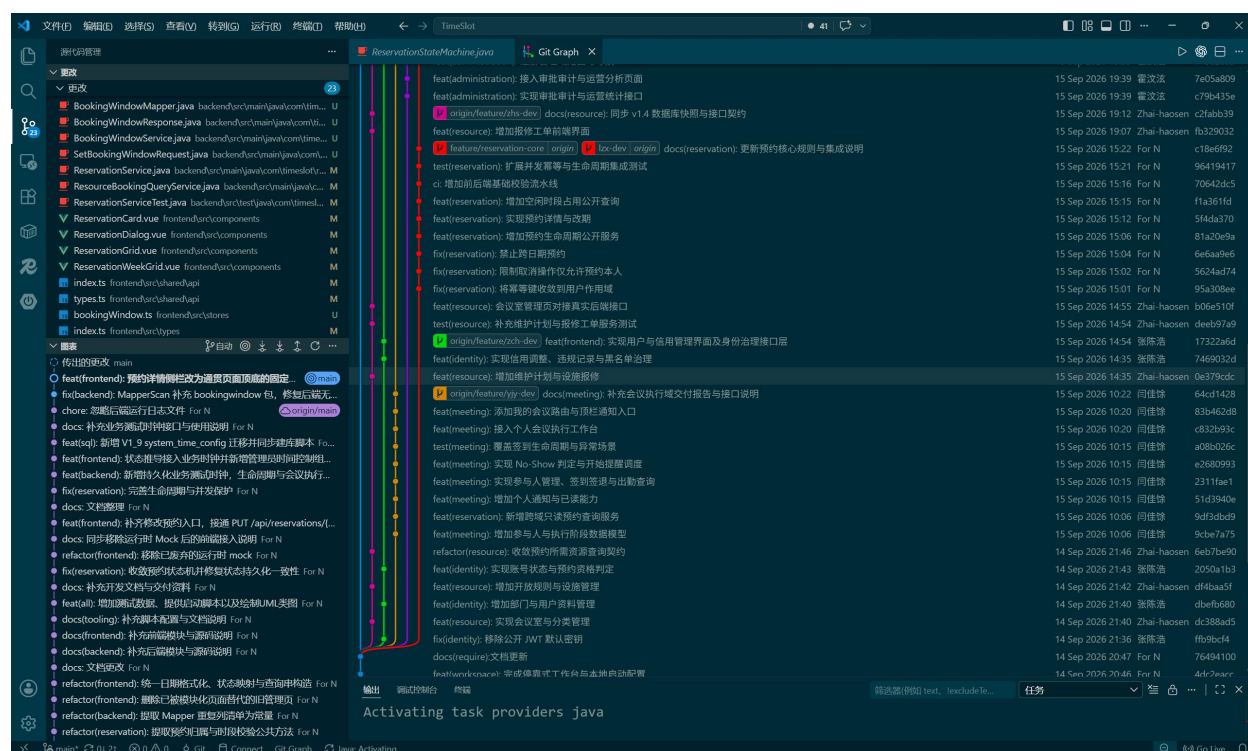
- 对外只读接口：为 meeting 域提供预约详情/日历查询，为 administration 域暴露审批端口；
- 前端看板闭环：实现预约看板、新建/修改对话框、预约详情、我的预约列表。

1.2 跨域边界与约束

按团队约定的五域边界，reservation 域只写本域 reservation 表，不直接操作 sys_user、meeting_room、approval_record、operation_log 等其它域的表。创建预约前通过 identity 域的 BookingQualificationService.check(userId) 进行只读资格校验；会议室可预约性、容量与开放窗口通过 resource 域的 ResourceBookingQueryService 只读查询，并通过 lockBookableRoom(roomId) 对会议室行加锁以获得串行化的冲突检测窗口；administration 域的审批/驳回/强制取消统一调用 reservation 域的 ReservationLifecycleService 端口，由该服务作为状态转换的唯一写入入口，避免状态机被绕开。

二、课程设计开始以来所做的具体工作和取得的进展

以下从交付范围、预约创建与并发控制、冲突检测算法、状态机与生命周期、前端看板、测试验证六个方面说明具体工作，并配以系统界面截图作为完成证明。



图：项目分工开发

2.1 交付范围总览

预约核心域全部能力均为真实后端实现，前端看板与预约表单也已接入后端。需求优先级与完成情况如下表所示：

优先级	能力	状态	说明
P0	预约创建（含资格、容量、开放时间、冲突校验）	已完成	ReservationService.createReservation
P0	预约改期（拥有人可改，重新走规则链）	已完成	ReservationService.update
P0	预约取消（普通用户取消）	已完成	ReservationService.cancel
P0	时间冲突检测（半开区间重叠）	已完成	countConflicts / TimeInterval.overlaps
P0	并发控制（会议室行锁 + requestId 幂等）	已完成	lockBookableRoom / findByUserIdAndRequestId
P0	看板日历查询	已完成	GET /api/reservations/calendar
P0	我的预约列表与详情	已完成	GET /api/reservations/my、/{id}
P0	预约生命周期（审批/驳回/强制取消）	已完成	ReservationLifecycleService
P1	前端预约看板与表单	已完成	ReservationBoard.vue / ReservationDialog.vue
P1	前端我的预约与详情	已完成	MyReservations.vue / ReservationDetail.vue

2.2 系统登录与预约看板入口

系统登录由身份认证域（identity）统一提供 BCrypt + JWT 鉴权。用户登录后默认进入「预约看板」页面，可直接查看会议室×时间的日视图网格，并切换日/周视图、按会议室筛选、仅显示空闲会议室。

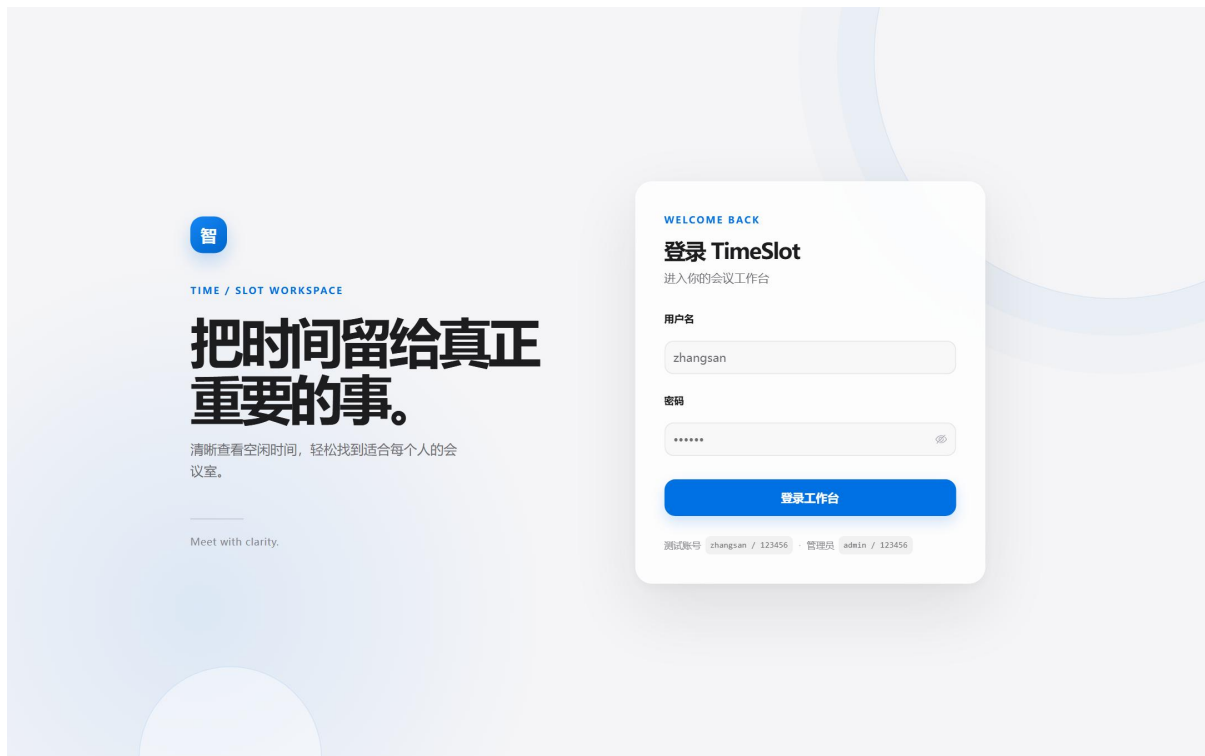


图 1 系统登录入口 (BCrypt + JWT 鉴权)

2.3 预约创建与并发控制

创建预约的入口为 POST /api/reservations。后端

ReservationService.createReservation 方法采用 READ_COMMITTED 事务，按如下顺序处理：

- 先以 requestId + 当前用户 id 查询是否已存在记录，若存在则直接返回已有预约（接口级幂等）；
- 调用 identity 域 BookingQualificationService.check 校验账号禁用/限制期/信用分；
- 解析并校验未来时间区间，拒绝跨日期、结束时间不晚于开始时间、早于当前时间等非法输入；
- 通过 resource 域 ResourceBookingQueryService.lockBookableRoom(roomId) 对 meeting_room 行加锁；
- 加锁后再次按 requestId 查询，避免并发重试在锁等待后得到误报的冲突；
- 执行完整规则链校验：会议室 AVAILABLE 状态、提前预约天数、最大预约时长、参会人数 ≤ 容量、开放窗口、半开区间冲突；
- 生成预约编号、持久化记录；状态根据会议室是否需要审批置为 PENDING 或 CONFIRMED。

会议室行锁 + requestId 幂等 + 锁后重查的组合，既防止了并发下的重复预约，也避免同一合法请求因网络重试被判定为冲突。

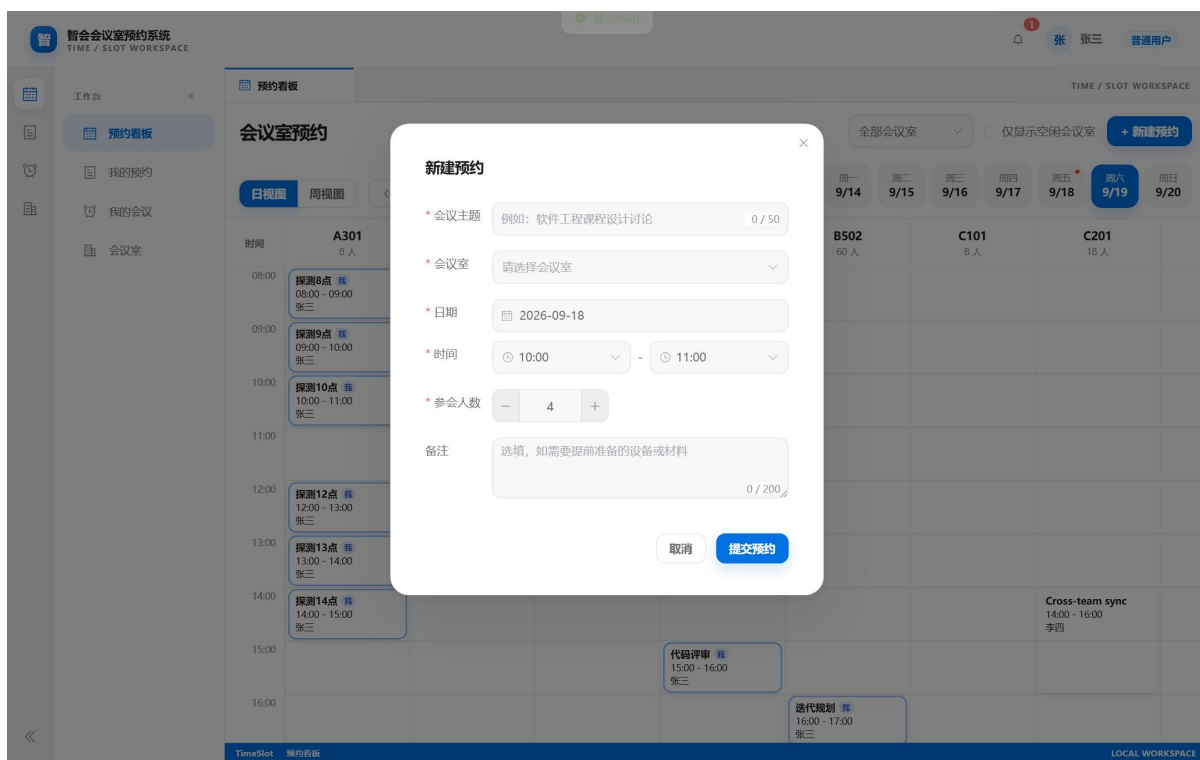


图 2 新建预约对话框（会议主题 / 会议室 / 日期 / 时间 / 参会人数 / 备注）

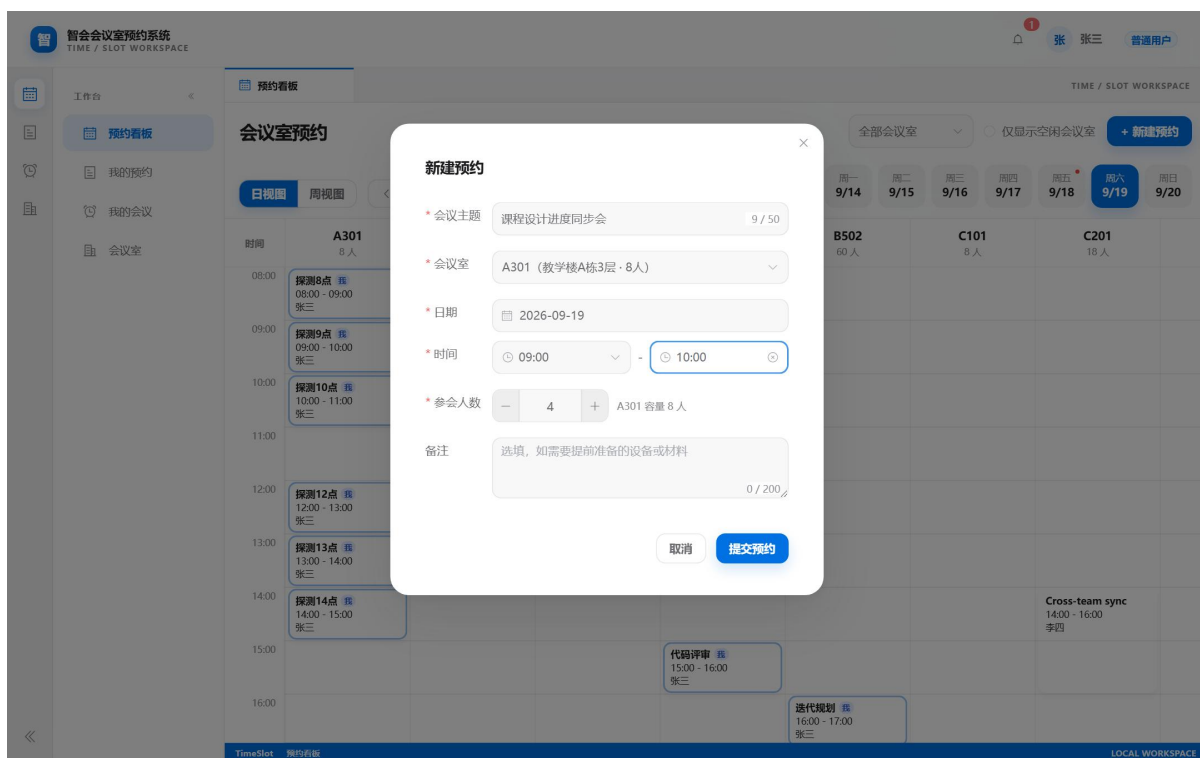


图 3 填写预约信息（以 A301 2026-09-19 09:00-10:00 为例）

2.4 半开区间时间冲突检测

冲突检测同时依赖数据库 SQL 与领域对象 `TimeInterval`。数据库层面，`ReservationMapper.countConflicts` 仅统计同一会议室、处于 `PENDING` 或 `CONFIRMED` 状态、且满足半开区间重叠条件 (`start_time < #{endTime}` AND `end_time > #{startTime}`) 的记录数。这一条件等价于半开区间 `[start, end)` 与 `[other.start, other.end)` 存在交集。

`TimeInterval` 值对象进一步在内存层做了同样的重叠判定：

```
public boolean overlaps(TimeInterval other) {  
    return start.isBefore(other.end()) && end.isAfter(other.start());  
}
```

当用户提交与已有预约重叠的时段时，后端返回 `409 RESERVATION_TIME_CONFLICT`；前端对话框保留表单并展示「预约失败：时间冲突」提示，同时给出可用会议室推荐或提示调整时间。

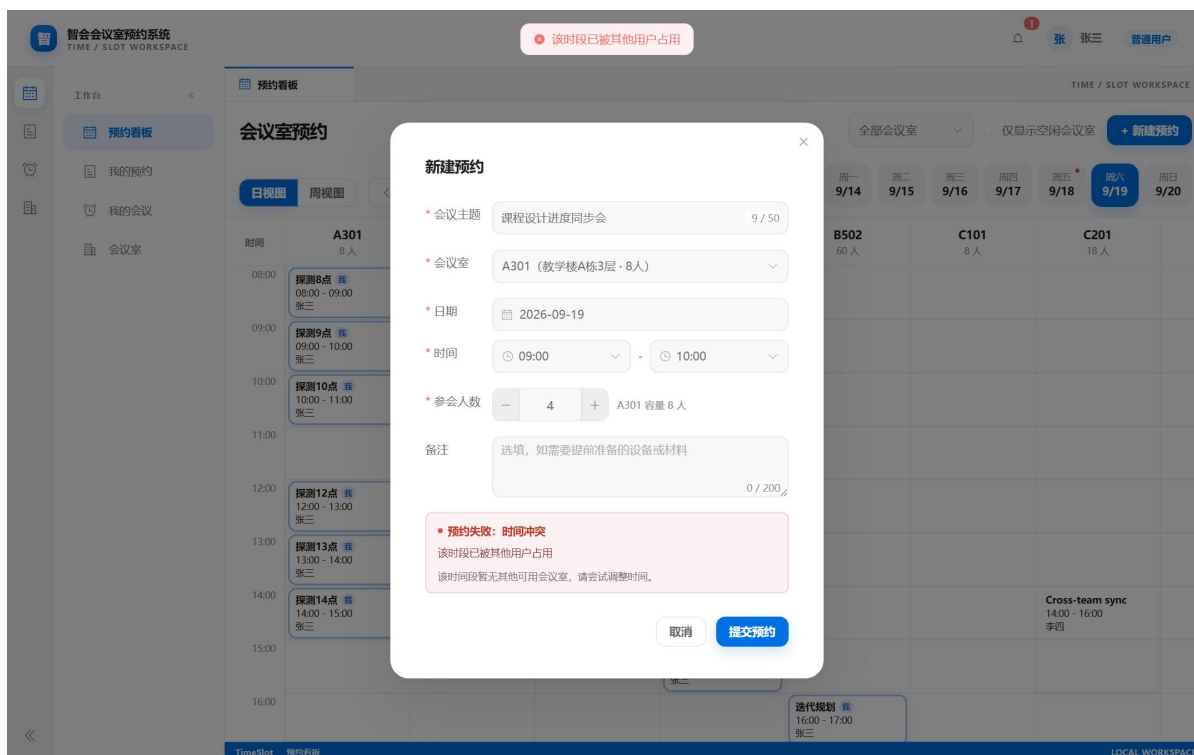


图 4 时间冲突拦截 (A301 09:00-10:00 已被占用)

2.5 预约状态机与生命周期

预约状态枚举为 `PENDING`、`CONFIRMED`、`REJECTED`、`CANCELLED`。状态转换写操作全部集中在 `ReservationLifecycleService`：

- PENDING → CONFIRMED: 管理员审批通过;
- PENDING → REJECTED: 管理员驳回, 必须填写原因;
- PENDING/CONFIRMED → CANCELLED: 管理员强制取消 (带原因), 或普通用户取消本人预约;
- CANCELLED/REJECTED 不可再回到其它状态。

任何其它域 (administration、meeting) 都必须通过 ReservationLifecycleService 的端口方法修改状态, 不能绕过该服务直接调用 ReservationMapper.updateStatus, 以保证状态机不被破坏。状态转换前使用 findByIdForUpdate 对预约行加锁, 避免并发审批/取消导致状态不一致。

2.6 预约看板、详情与我的预约

前端 ReservationBoard.vue 以日视图展示「会议室 × 时间」网格, 每条预约以 .res-card 卡片渲染, 不同状态使用不同颜色 (confirmed 蓝色、pending 橙色), 当前用户预约带有「我」标识。点击卡片打开 ReservationDetail.vue 详情弹窗, 展示会议主题、会议室、日期、时段、状态、备注等信息; 具备权限的用户可执行取消或改期。

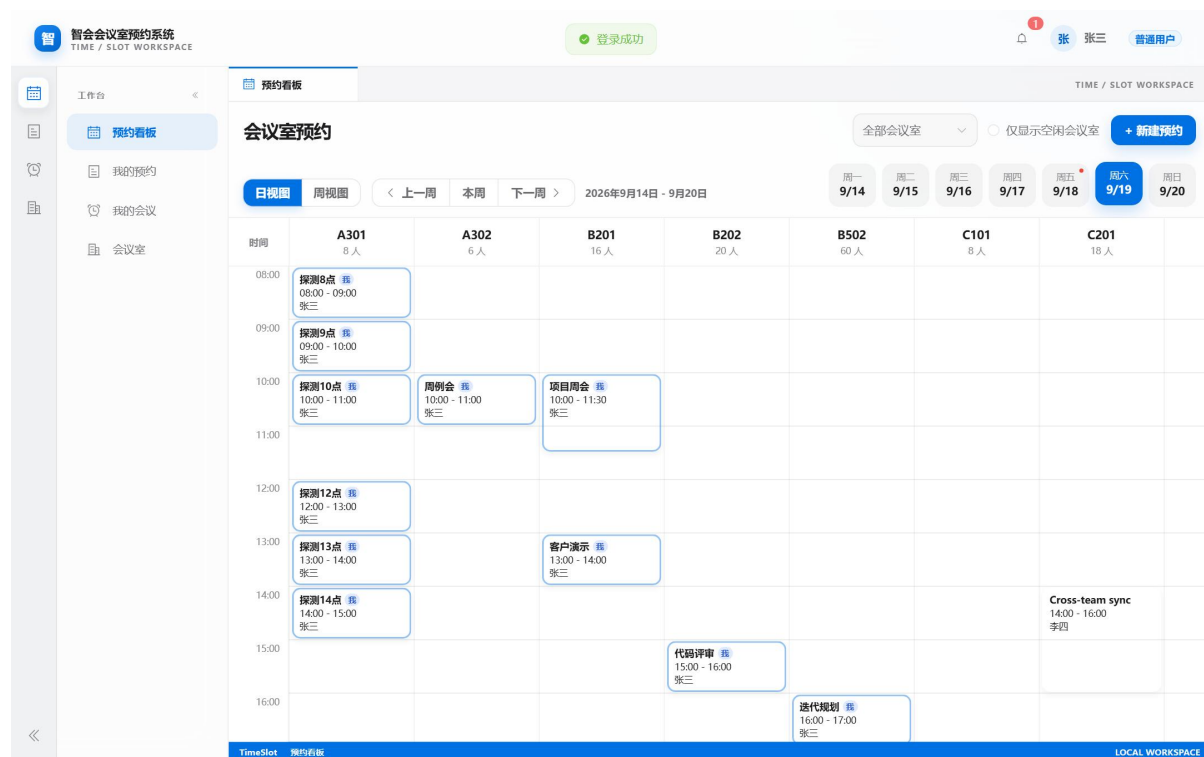


图 5 预约看板日视图 (2026-09-19 多会议室预约分布)

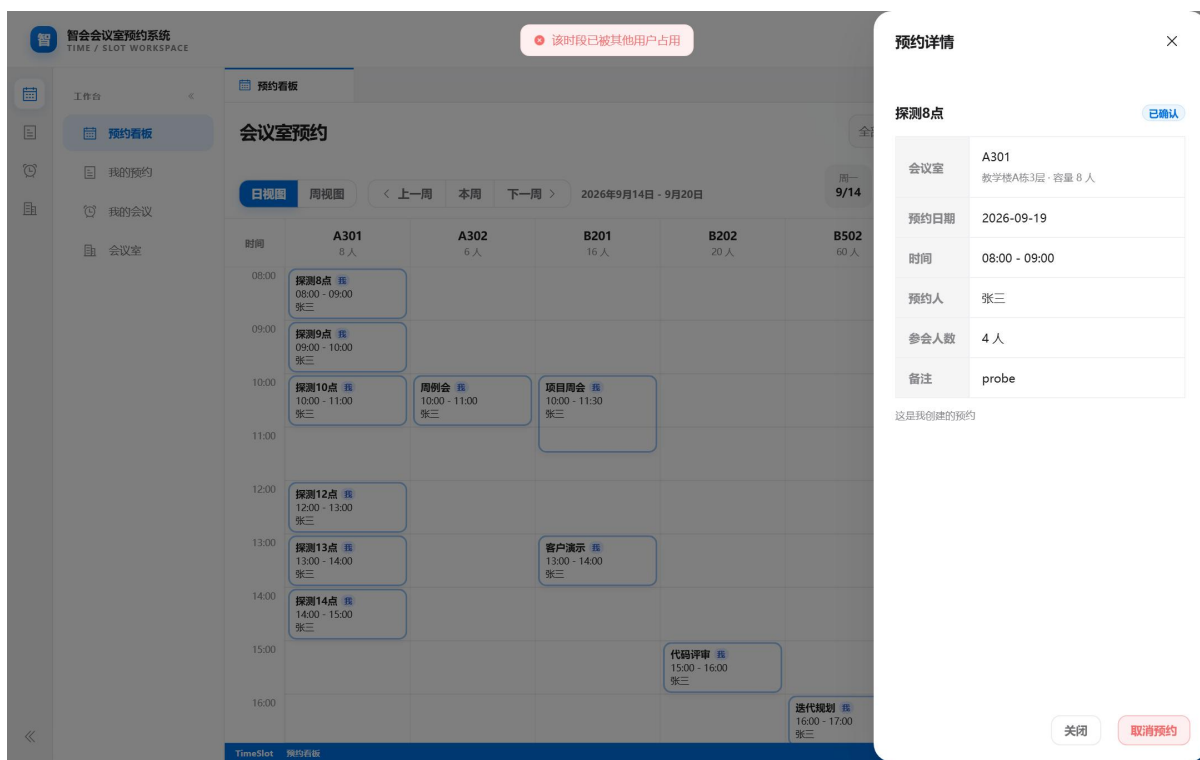


图 6 预约详情弹窗

MyReservations.vue 提供「我的预约」列表，支持按状态（全部/待进行/进行中/已结束/已取消/已驳回）筛选，列表展示会议主题、会议室、日期、时间、状态和操作入口，并提供「信用记录」快捷入口。

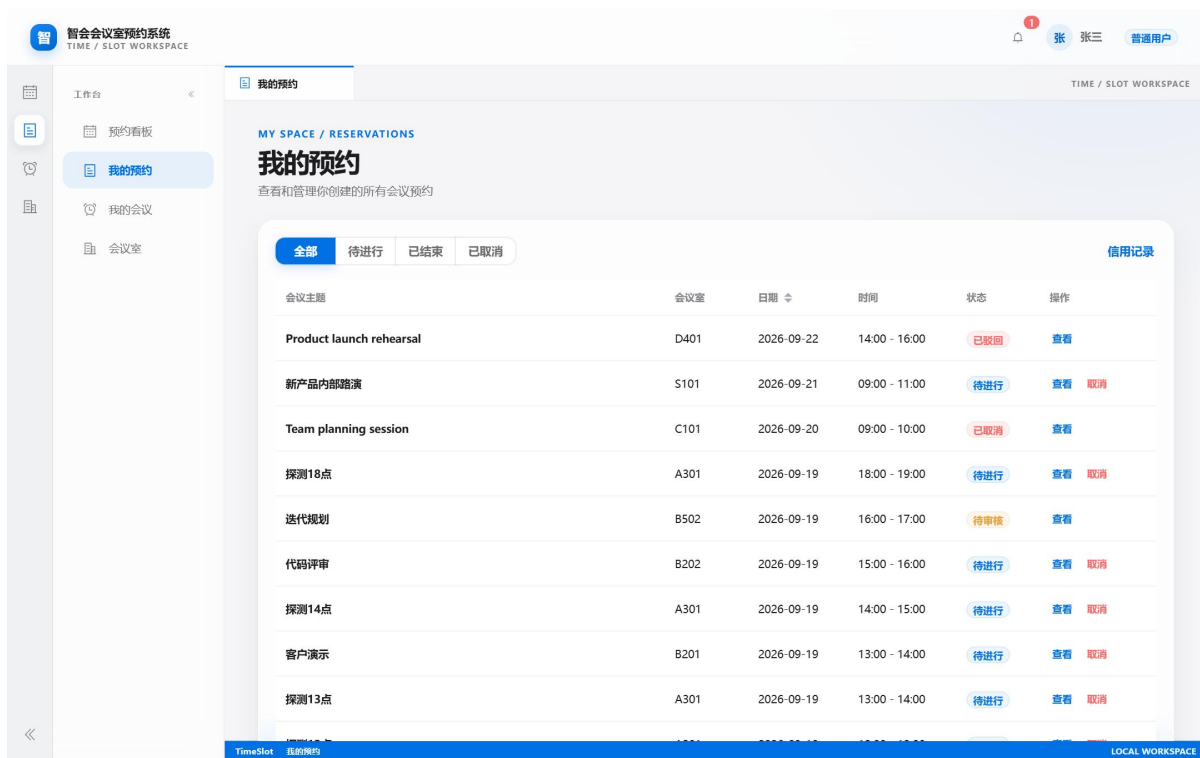


图 7 我的预约列表

2.7 关键接口与测试验证

预约核心域对外提供的关键 REST 接口如下表所示：

Method	Path	权限	说明
POST	/api/reservations	登录用户	创建预约，返回 201 或 409 冲突
PUT	/api/reservations/{id}	预约本人	修改预约时段/会议室
DELETE	/api/reservations/{id}	预约本人	取消本人预约
GET	/api/reservations/calendar	登录用户	日历/看板查询 (start/end/roomId)
GET	/api/reservations/my	登录用户	我的预约列表
GET	/api/reservations/{id}	登录用户	预约详情

后端 reservation 域共 52 个自动化测试，全部通过：

- ReservationServiceTest (25 例)：覆盖创建成功、时间冲突、容量超限、开放时间外、跨日期、资格不足、requestId 幂等、本人改期/取消、非本人越权等；
- ReservationLifecycleServiceTest (9 例)：覆盖审批、驳回、强制取消、状态机非法转换、原因必填等；

- ReservationConcurrencyIT (7 例)：覆盖并发创建冲突、并发重复 requestId、锁等待后幂等返回等；
- TimeIntervalTest (7 例)：覆盖相离、相交、相接、相等、跨午夜、无效区间等半开区间场景；
- ReservationQueryServiceTest (4 例)：覆盖日历查询、我的预约列表、详情访问权限。

2.8 提交记录

本人在 reservation 核心域共有 16 个单一职责 commit，并已推送 origin。以下列出具有代表性的 8 个：

Commit	说明
30b194e	feat(reservation)：增加预约生命周期公开服务
b7e8149	fix(reservation)：完善生命周期与并发保护
badel54	fix(reservation)：收敛预约状态机并修复状态持久化一致性
328944e	fix(reservation)：限制取消操作仅允许预约本人
5757b07	refactor(reservation)：提取预约归属与时段校验公共方法
95a308e	fix(reservation)：将幂等键收敛到用户作用域
9641941	test(reservation)：扩展并发幂等与生命周期集成测试
97adf60	feat(frontend)：补齐修改预约入口，接通 PUT /api/reservations/{id}

三、目前存在问题，下一步的主要设计任务，具体设想与安排

目前存在问题	下一步任务	具体设想与安排
当前时间冲突以会议室维度判定，尚未支持「同一用户同一时间只能预约一个会议室」等全局约束。	增加用户-时间唯一性校验	在 createReservation 的 validateSlot 前增加按 user_id + 时间范围的冲突查询，返回 RESERVATION_USER_TIME_CONFLICT。
改期接口尚未接入前端的日历网格拖拽/点击改期交互，仅通过详情页入口可用。	看板直接改期交互	在 ReservationGrid 组件中暴露 res-card 的右键/拖拽事件，将新时段作为 initial 打开 ReservationDialog 并调用 PUT。

会议室开放窗口目前按工作日统一配置，节假日与特殊日期尚未覆盖。	接入业务时钟与特殊日期配置	读取 V1_9 system_time_config 中的业务时钟与节假日表，在 validateSlot 的开放窗口校验中跳过节假日。
强制取消缺少审计记录联动，operation_log 与 approval_record 的写入分散在 administration 域。	统一生命周期审计事件	ReservationLifecycleService 状态转换后发布领域事件 ReservationStatusChangedEvent，由 administration/monitor 订阅并落库。
前端冲突提示的推荐会议室列表在复杂多冲突场景下排序不够智能。	优化可用会议室推荐策略	按距离原会议室楼层最近、容量最接近 participantCount、时间完全包含等规则对 alternatives 排序。

经过开发、自测与团队评审，预约核心域仍存在以下待完善项：

综上所述，预约调度与冲突控制核心域已完成预约创建、改期、取消、半开区间冲突检测、并发控制、状态机、对外只读接口与前端看板等全部 P0 能力，并提供了 52 个自动化测试保障。后续将优先补齐用户-时间唯一性约束与看板直接改期交互，再逐步接入业务时钟与统一审计事件机制。

一、本人在该设计中具体应完成的工作（翟浩森）

姓名	翟浩森
学号	2413042116
班级	软件工程 / 计算机学院
负责模块	资源管理领域的设计与实现

1.1 职责范围概述

智会会议室预约系统 (TimeSlot) 采用前后端分离架构, 后端基于 Spring Boot 3 + MyBatis + MySQL, 前端基于 Vue 3 + Vite, 按业务划分为身份、资源、预约、会议、行政五个领域。本人在小组中负责会议室资源管理域 (resource) 的完整设计与实现, 该领域是整个预约业务的数据基础: 系统中所有的预约看板、预约下单和会议执行, 都必须依赖资源域提供的会议室档案、开放时间规则和并发加锁服务。

本人承担的工作贯穿数据库设计、后端接口开发、前端页面开发、单元测试与文档同步五个环节, 具体任务分解如表 1 所示。

表 1 本人承担的设计任务与产出物

任务领域	具体工作内容	主要产出物
数据库设计	设计会议室分类、会议室、设施、开放规则、维护计划、报修工单六张表的结构与约束, 并编写 v1.4 增量迁移脚本	sql/schema.sql、 V1_4__resource_management.sql
后端—会议室与分类	会议室新建、编辑、状态切换, 分类规则 (容量区间、审批要求、最长时长、可预订天数) 的维护	RoomAdminController、 RoomAdminService、 CategoryAdminService 等
后端—开放时间 与设施	每周开放时段规则的整批保存, 会议室设施清单的维护与展示	replaceOpenRules、 replaceFacilities 接口
后端—维护与报修	维护计划的登记、查询与完结; 用户报修工单提交、管理员工单列表与处理	RoomMaintenanceService、 RepairTicketService
后端—预约域契约	向预约领域提供会议室事务行锁与开放时 间窗口查询, 隔离两域的直接依赖	ResourceBookingQueryService、 BookableRoomProfile
前端页面	会议室列表页、会议室管理后台、报修工单管理页的开发与真实接口联调	MeetingRooms.vue、 RoomAdmin.vue、 RepairAdmin.vue

任务领域	具体工作内容	主要产出物
单元测试	维护计划与报修工单两个服务的单元测试，覆盖正向流程、校验失败与并发场景	RoomMaintenanceServiceTest（7例）、RepairTicketServiceTest（8例）
文档同步	同步 v1.4 数据库快照与资源域接口契约文档	docs 接口契约与数据库设计文档

1.2 与其他成员的协作边界

为降低各领域之间的耦合，小组约定：预约域不允许直接访问资源域的数据库表和数据库访问层，只能通过本人实现的 `ResourceBookingQueryService` 获取会议室信息。预约事务在创建前必须先调用该服务的加锁方法锁定会议室行，从而保证同一会议室在并发预约时不会被超额预订。该契约由本人在重构中收敛完成，并以专门的数据传输对象（`BookableRoomProfile`、`BookingOpenWindow`）对外暴露，预约域只依赖这两个类而不是数据库实体。

二、课程设计开始以来所做的具体工作与取得的进展

2.1 开发过程与提交记录

本人的开发工作在独立功能分支 `feature/resource-management` 上进行，共形成 8 次有效提交，累计新增代码约 3000 行。开发顺序上先后端后前端：先完成数据库与服务层能力，再完成管理页面对接，最后补充测试与文档。全部工作已于 9 月 15 日前后合入主分支 `main`。具体提交记录如表 2 所示。

表 2 本人的代码提交记录（git log）

日期	提交说明	变更规模
09-14	feat(resource): 实现会议室与分类管理	16 个文件, +606 / -26
09-14	feat(resource): 增加开放规则与设施管理	11 个文件, +265 / -1
09-14	refactor(resource): 收敛预约所需资源查询契约	6 个文件, +114 / -71

日期	提交说明	变更规模
09-15	feat(resource): 增加维护计划与设施报修	11 个文件, +503 / -2
09-15	test(resource): 补充维护计划与报修工单服务测试	2 个文件, +323
09-15	feat(resource): 会议室管理页对接真实后端接口	4 个文件, +785 / -54
09-15	feat(resource): 增加报修工单前端界面	4 个文件, +314 / -1
09-15	docs(resource): 同步 v1.4 数据库快照与接口契约	3 个文件, +92 / -11

2.2 数据库结构设计

数据库层面本人设计了六张业务表：会议室分类表 `room_category` 存储分类名称、容量上下限、是否需要审批、单次预约最长时长与可提前预订天数；会议室表 `meeting_room` 通过外键关联分类，状态字段以整数存储（1 可用、0 维护中、2 停用）；设施表 `room_facility` 与开放规则表 `room_open_rule` 分别以会议室为维度记录设备清单和每周每日的开放时段；维护计划表 `room_maintenance` 记录维护时间段与状态；报修工单表 `facility_repair_ticket` 记录报修对象、故障描述、报修人与处理结果。名称唯一性由数据库唯一索引保障，作为应用层查重之外的最后一道防线。

2.3 后端服务实现

后端代码严格按控制器—服务—数据访问三层组织，全部 SQL 集中在 `ResourceMapper` 一个接口内，便于统一优化。控制器只负责参数校验、权限注解与统一响应封装；业务规则全部收敛在服务层。实现中的几个关键技术点如下。

（1）状态枚举与数据库映射。会议室状态定义为枚举 `MeetingRoomStatus`（可用、维护中、停用），并在枚举内实现与数据库整数的双向转换，非法值直接招错。业务代码始终面向枚举，数据库的存储细节不会渗透到业务逻辑中。

（2）面向预约域的事务行锁。`ResourceBookingQueryService` 的加锁方法使用 `SELECT ... FOR UPDATE` 锁定会议室记录，并通过事务传播级别 `MANDATORY` 强制要求调用方已处于事务内，从而保证锁的生命周期与预约事务一致，事务提交前锁不释放，从机制上避免了超额预订。

(3) **数据一致性防护**。会议室与分类的名称查重采用先查询后插入加数据库唯一索引兜底的双保险方式，并发提交重名时捕获重复键异常转化为友好提示；开放规则保存前校验关闭时间必须晚于开放时间且每个星期仅一条规则；报修工单的处理操作在 UPDATE 语句中附加状态条件，根据影响行数判断是否处理成功，防止并发重复处理。

(4) **查询性能优化**。会议室列表使用 GROUP_CONCAT 将每间房间的设施名在一条 SQL 内拼接返回，避免了遍历房间逐个查询设施的 N+1 查询问题。

2.4 前端页面实现与接口联调

前端共开发三个页面：面向所有用户的会议室列表页，展示容量、设施与实时空闲状态，并提供设施报修入口；面向管理员的会议室管理页，支持新增、编辑、状态切换、设施配置、开放时间配置与维护计划登记；面向管理员的报修工单页，未处理工单置顶展示，支持按会议室筛选与处理回填。三个页面均已对接真实后端接口，不再使用模拟数据。

2.5 系统运行效果与截图证明

以下截图均摄于本地运行的完整系统（后端端口 8080，前端端口 5173，数据库为初始化后的真实种子数据）。图 1 为登录页，图 2 为登录后的预约看板，看板中展示的会议室列表即来自本人开发的资源接口；图 3 为会议室列表页，展示容量、设施标签与实时空闲状态。

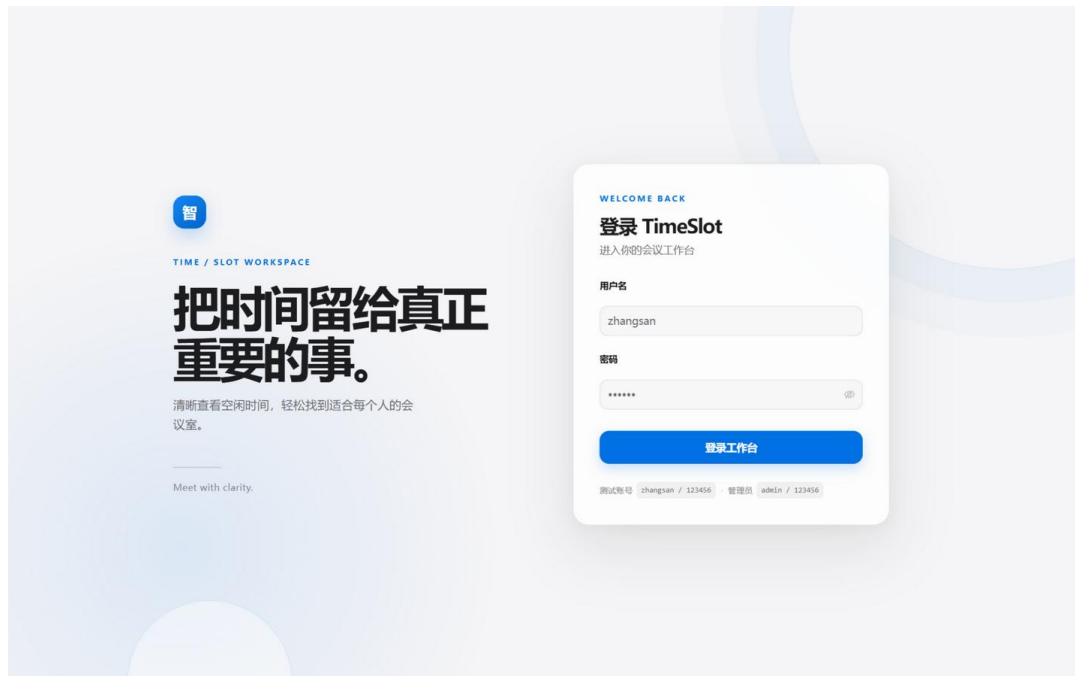


图 1 系统登录页面

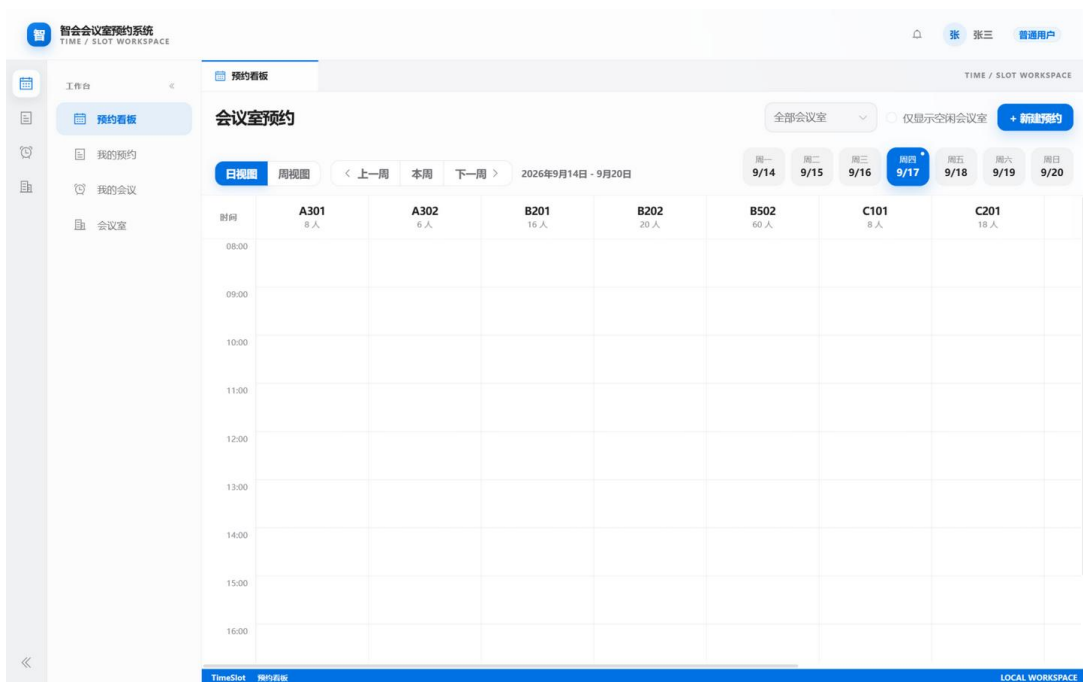


图 2 登录后的预约看板（会议室数据来自资源接口）

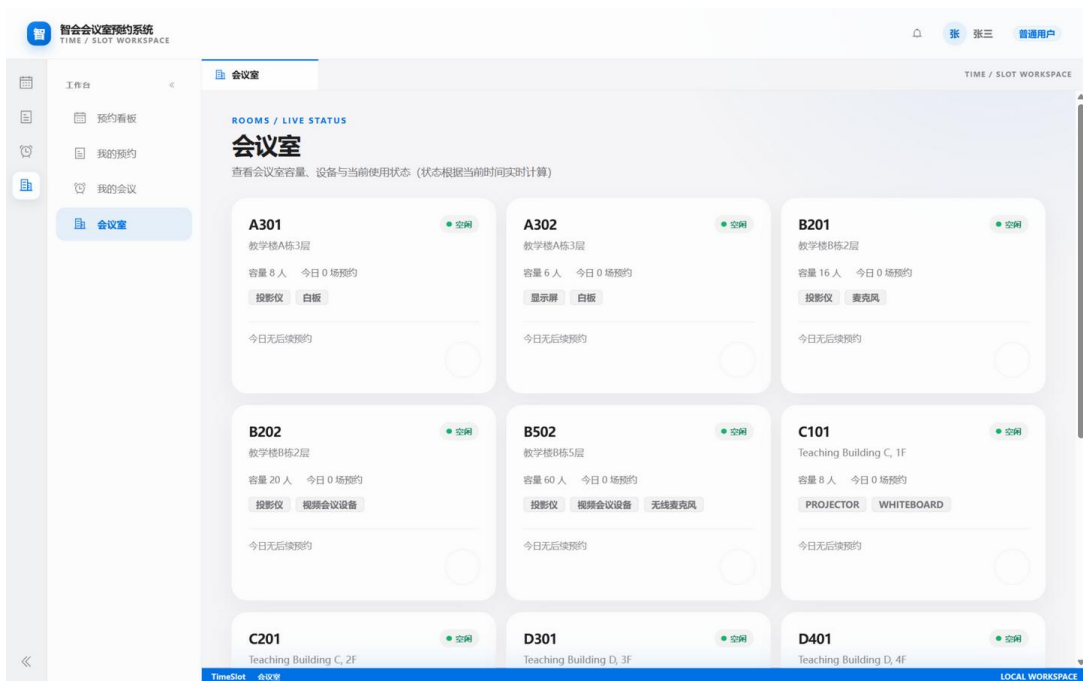


图 3 会议室列表页（用户视角）

图 4 为管理员登录后的会议室管理页，表格展示名称、位置、容量、分类、设施与状态，每行提供编辑、设施、开放时间、维护、状态五项操作；图 5 为 A301 会议室的每周开放时间配置弹窗，可按星期勾选并设置时段，页面同步提示了跨日开放暂不支持的业务规则。

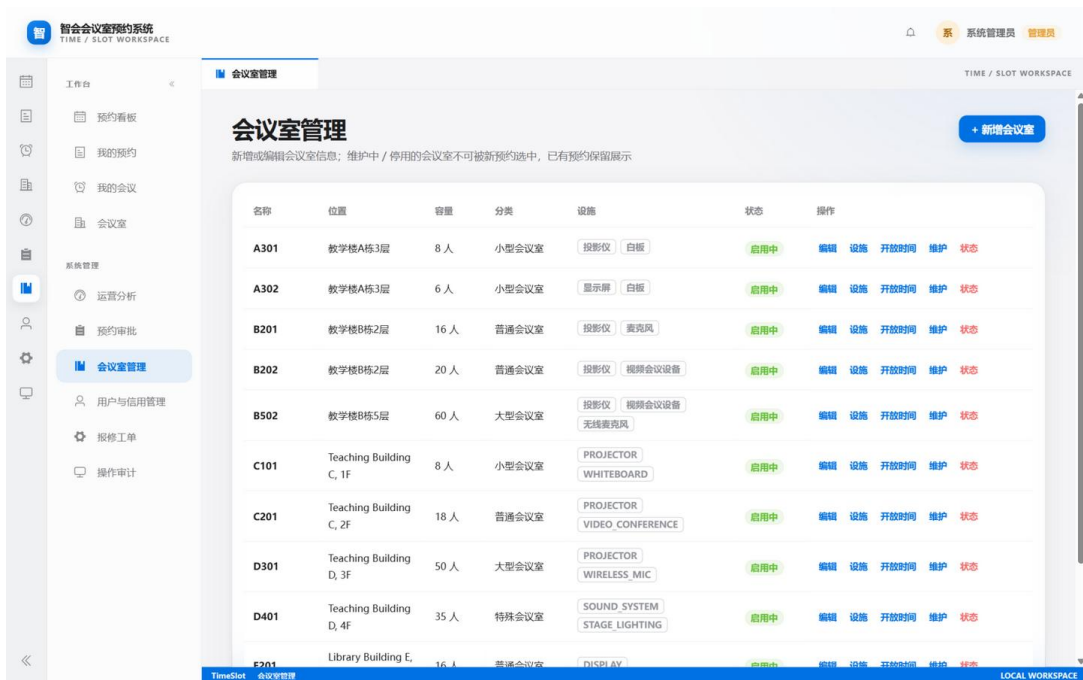


图 4 会议室管理页（管理员视角）

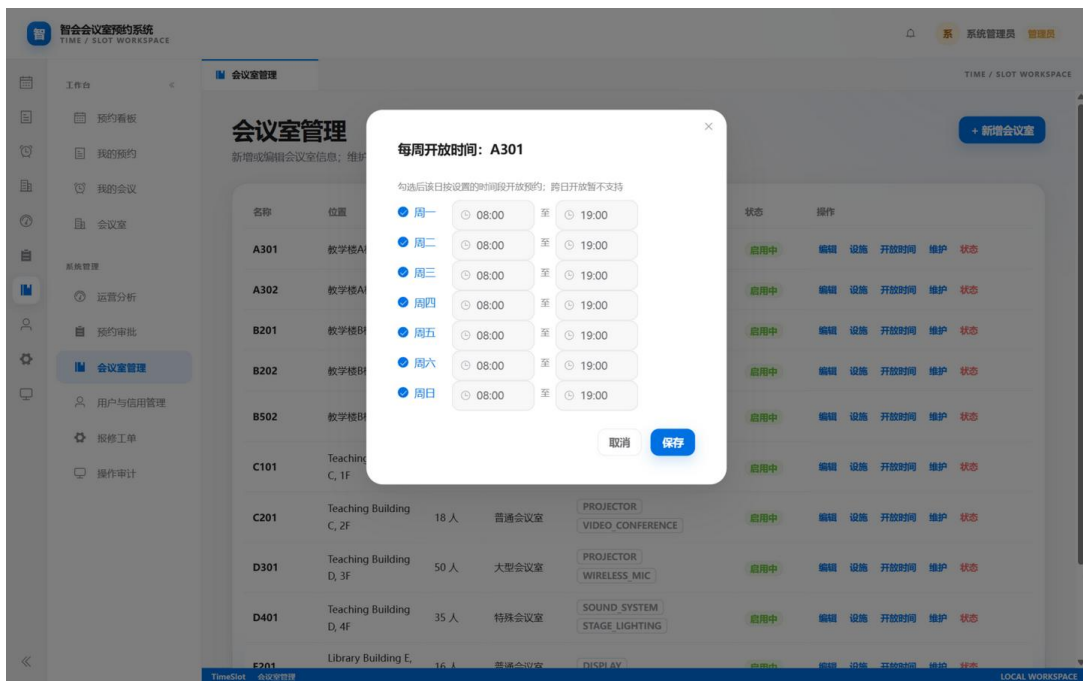


图 5 每周开放时间配置弹窗

2.6 报修工单全流程演示

为验证报修业务的完整闭环，演示了从用户提交到管理员处理的全过程。图 6 为普通用户在 A301 会议室卡片上发起设施报修，选择报修对象投影仪并填写故障描述；提交后系统提示报修已提交。图 7 为管理员端的报修工单列表，刚提交的工单以待处理状态

置顶展示；图 8 为管理员点击解决后填写处理说明；图 9 为处理完成后的列表状态，工单变为已解决并保留处理说明与时间，且同一工单不可再次处理。

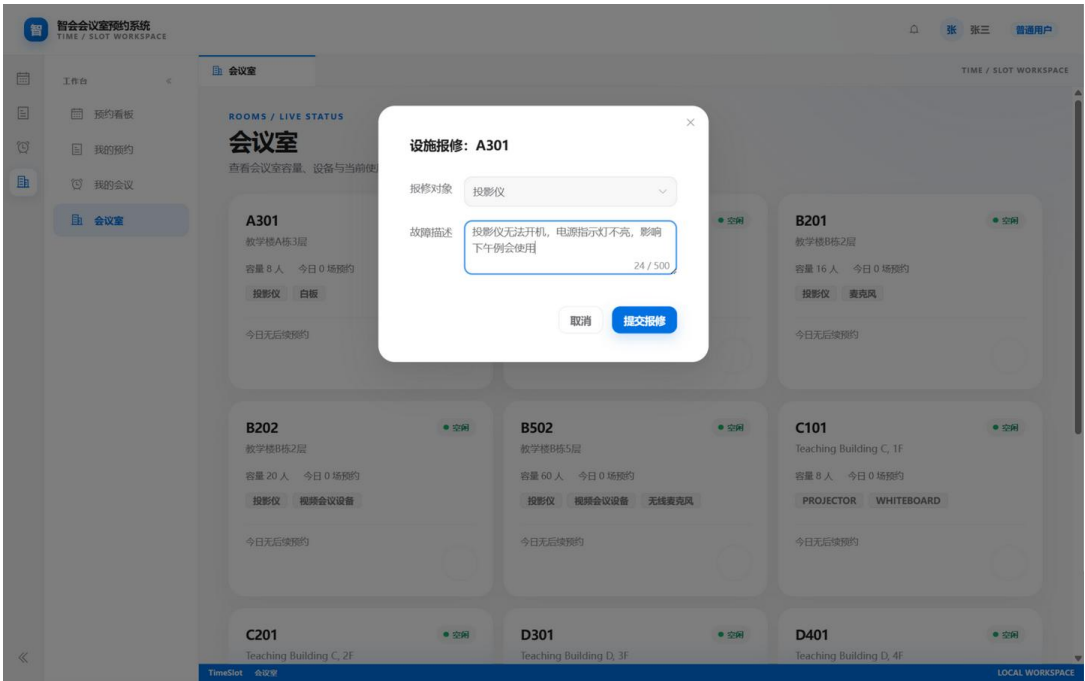


图 6 用户提交设施报修（A301 投影仪）

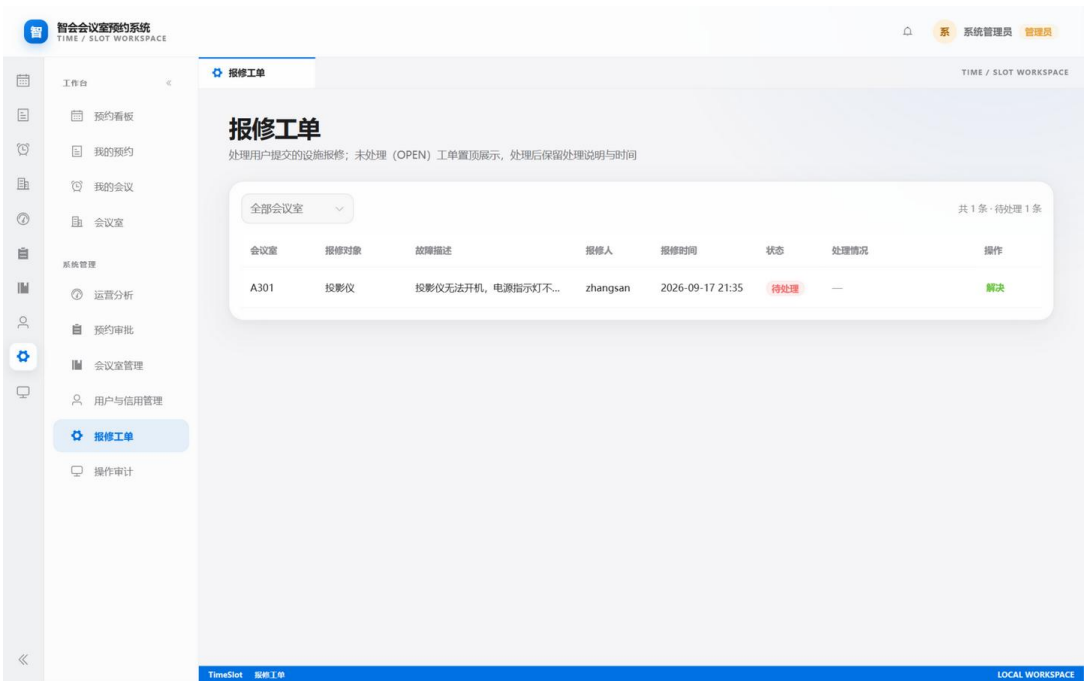


图 7 报修工单管理页（待处理置顶展示）

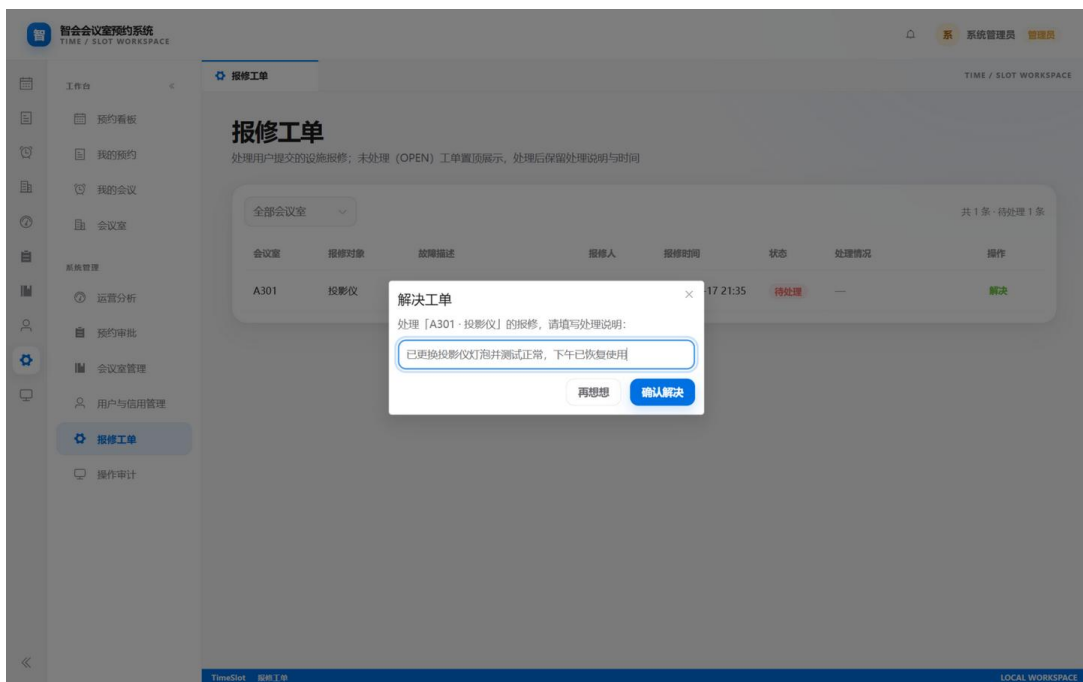


图 8 管理员填写处理说明



图 9 工单处理完成（已解决，保留处理记录）

2.7 维护计划登记

管理员可在会议室管理页为任意会议室登记维护计划。图 10 为 A301 的维护计划登记弹窗：填写维护原因（投影仪年度检修保养）与起止时间后点击登记，计划即以计划中状态入库并展示在下方列表中，可在维护完成后点击结束；结束操作带状态校验，不允许

重复结束。弹窗同时提示：维护计划仅作登记，需要阻断预约请同时将会议室置为维护中，该提示对应第三节中已列出的改进任务。

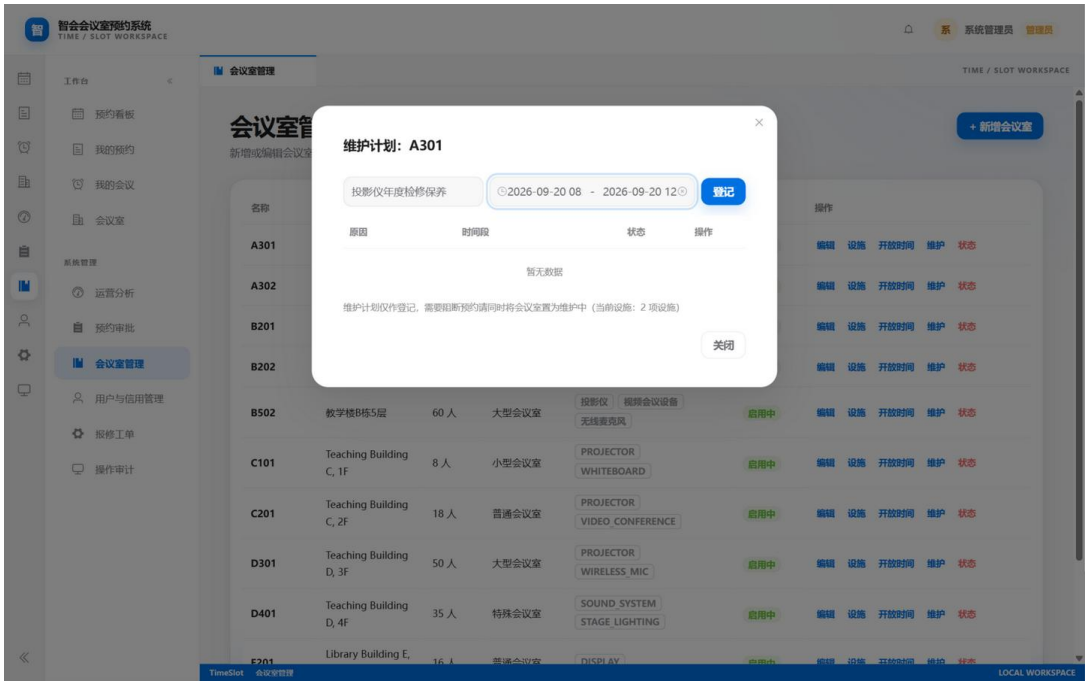


图 10 维护计划登记弹窗（填写维护原因与起止时间）

2.8 测试与验证工作

单元测试方面，本人为维护计划与报修工单两个核心服务编写了 15 个测试用例，采用 JUnit 5 与 Mockito 对数据访问层打桩，覆盖正向流程、参数校验失败、越权操作（如设施不属于指定会议室）以及并发重复处理等场景。在本地执行 mvn test 的结果为：RepairTicketServiceTest 运行 8 例、RoomMaintenanceServiceTest 运行 7 例，全部通过，无失败无错误。

接口联调验证方面，对启动后的系统逐个验证了登录、会议室列表、分类列表、报修工单列表等接口：登录接口正确签发 JWT 令牌，会议室列表接口返回 11 间会议室的真实数据，分类接口返回分类规则，报修提交与处理接口在页面操作中全部走通，数据库记录变化正确。

2.9 阶段性进展小结

截至目前，本人负责的资源管理域已完成设计、开发、测试、联调与文档全部计划内容：后端 35 个源文件、约 1580 行代码，前端 3 个页面全部对接真实接口，功能分支已合入主分支，系统可完整演示会议室查询、管理、报修与维护业务的完整闭环。

三、目前存在问题、下一步主要设计任务、具体设想与安排

3.1 目前存在的问题

在联调与代码走查中，本人梳理出当前实现中存在以下六个问题：一是会议室、工单、维护计划的列表接口均未做分页，数据量增大后响应时间与前端渲染压力会上升；二是会议室列表使用 GROUP_CONCAT 拼接设施，受 MySQL 默认 1024 字节上限约束，设施极多时可能被静态截断；三是分类名称查重目前将全部分类载入内存后遍历比对，未使用按名精确查询；四是分类列表接口挂在管理前缀下但未限制管理员角色，路由语义与权限控制不一致；五是维护计划仅作登记，不会自动将会议室置为维护中，需管理员手工切换状态才能阻断预约；六是开放时间规则不支持跨日开放（如晚间至次日清晨）。

3.2 下一步主要设计任务与安排

针对上述问题，结合答辩时间节点，制定了下一阶段的任务清单与时间安排，如表 3 所示。

表 3 下一步任务安排

优先级	任务	预期产出	计划完成时间
高	列表接口分页与条件筛选	会议室、工单、维护计划支持页码与状态筛选	第十八周
高	维护计划与预约阻断联动	创建维护计划时自动校验重叠预约并提示置状态	第十八周
中	分类查重与权限修正	按名精确查询；分类列表接口权限与路由前缀对齐	第十九周
中	设施查询方式优化	消除 GROUP_CONCAT 长度限制风险	第十九周
低	开放时间跨日支持	窗口模型支持跨天区间并同步预约校验	答辩后持续迭代

3.3 具体技术设想

（1）分页改造。在 Mapper 层查询语句中增加 LIMIT 与偏移参数并返回总数，前端使用分页组件展示；工单列表保留未处理置顶排序规则，仅在分页内排序。

(2) **维护与预约联动**。创建维护计划时在同一事务内查询时间段重叠的未完成预约，存在冲突时拒绝登记并提示具体冲突时段；登记成功时自动将会议室置为维护中，结束计划时提示恢复为可用，使页面提示的手工操作变为系统保障。

(3) **查询与权限修正**。为分类表增加按名称的精确查询替换全表遍历；将分类列表接口调整到非管理前缀或补充管理员鉴权，使路由与权限语义一致。

(4) **设施查询优化**。将会议室列表拆为两次查询在服务层内存组装，或调大 `group_concat_max_len` 并在文档中标注约束，两种方案中选择对上层无感知的后者作为短期方案。

(5) **跨日开放**。开放窗口数据结构增加跨天标识，预约校验时允许结束时间小于开始时间的窗口按次日拼接，同步修改前端表单校验与提示。

3.4 后续学习与答辩准备

除代码迭代外，接下来将重点做三件事：一是整理资源域的设计文档与接口契约，准备答辩中关于并发控制、事务边界与模块解耦的问答提纲；二是配合预约域同学完成行锁接口的联合压测，验证并发预约场景下的数据一致性；三是补充系统的部署说明，整理数据库初始化、环境变量配置与启动脚本的使用步骤，确保答辩现场可一键启动演示。

课程设计个人报告

会议室预约与时间冲突检查系统 • 会议执行域

一、本人在该设计中具体应完成的工作（闫佳馥）

姓名	闫佳馥
学号	2413042121
班级	24130421
负责模块	会议执行域（meeting）
指导教师	张淑娟
团队名称	尖草坪游击队

在本次课程设计中，本人负责「会议室预约与时间冲突检查系统」中的会议执行域（meeting domain）。会议执行域是预约流程之后的关键闭环：用户预约成功后，需要管理参与人、在会议时间窗口内完成签到/签退、由系统自动判定缺席（No-Show）并触发信
用联动，同时为普通用户提供「我的会议」个人工作台与业务内通知。

1.1 功能职责

- 参与人管理：为已确认的预约添加/移除参与人，自动幂等补齐组织者行，校验参与人上限与权限边界；
- 签到/签退：基于服务器时间提供 [开始-15 分钟, 结束) 签到窗口，以及结束+60 分钟宽限内的签退窗口，支持幂等调用；
- No-Show 判定：会议结束后由调度器自动将未签到用户标记为缺席，保证幂等可重复执行；
- 开始提醒：会议开始前向参与人与组织者推送开始提醒通知；
- 个人通知中心：提供被加入/移出会议、开始提醒、No-Show 结果等通知的列表、未读数与已读能力；
- 前端闭环：实现「我的会议」工作台，覆盖阶段筛选、签到/签退、参与人抽屉、出勤历史与通知入口。

1.2 跨域边界与约束

按团队协作约定，会议执行域仅可写本域事实表（reservation_attendee、notification），对 reservation/sys_user/meeting_room 等表仅做只读 JOIN；不修改 identity、reservation、administration、resource 等其它域的表结构与 Mapper，跨域集成点通过新增只读服务（ReservationQueryService）或预留 SPI 回调完成。

二、课程设计开始以来所做的具体工作和取得的进展

以下从数据模型、后端服务、业务规则、前端闭环、测试验证与提交记录六个方面说明具体工作，并配以系统界面截图作为完成证明。

2.1 交付范围总览

会议执行域全部能力均为真实后端实现，无 Mock、无空表。需求优先级与完成情况如下表所示：

优先级	能力	状态	说明
P0	参与人管理（增/删/查，权限与人数约束）	已完成	创建人/管理员可管理，组织者不可移除
P0	签到/签退（窗口、幂等、服务器时间）	已完成	窗口外返回明确业务错误
P0	No-Show 判定（结束后调度、幂等、可重复执行）	已完成	每分钟扫描，仅影响 EXPECTED 行
P0	用户前端闭环（我的会议、参与人、签到签退、出勤历史、通知入口）	已完成	Vue3 + Element Plus
P1	业务内通知（被加入/移出、开始提醒、No-Show 结果）	已完成	支持列表、未读数、已读

2.2 数据模型与数据库迁移

新增 V1_5__meeting_execution.sql (owner: meeting)，从基线 v1.1 升级，未修改任何既有表。新增两张事实表：reservation_attendee（执行事实表）与 notification（个人通知）。其中 reservation_attendee 记录 attendee_role（ORGANIZER/ATTENDEE）、attendance_status（EXPECTED/CHECKED_IN/CHECKED_OUT/NO_SHOW）及 check_in_at/check_out_at，并以 (reservation_id, user_id) 唯一键防重。notification 表使用 uk_notification_dedup(user_id, dedup_key) + INSERT IGNORE 保证调度类通知幂等。

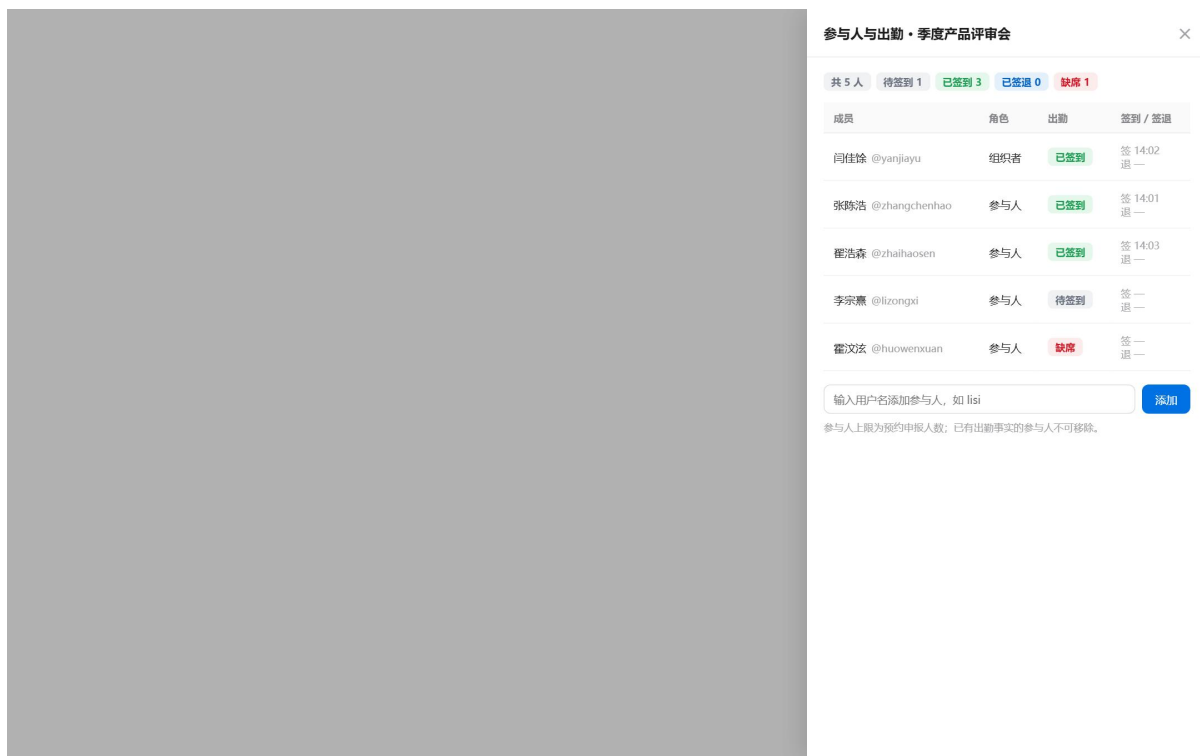


图 1 参与人与出勤管理抽屉（对应 `reservation_attendee` 表数据）

2.3 后端核心服务实现

后端入口为 `MeetingExecutionController`，统一返回 `ApiResponse<T>`。核心服务 `MeetingExecutionService` 完成以下逻辑：

- 参与人新增/移除：校验预约状态（仅 `CONFIRMED` 可改）、人数上限、权限、组织者保护；
- 签到/签退：使用 `Clock Bean` 获取服务器时间，校验窗口与状态机，重复调用返回当前事实（幂等）；
- 出勤查询：支持组织者/管理员/参与人三种可见性，返回出勤总览与当前用户出勤；
- 开始提醒调度：每分钟扫描 15 分钟内开始的会议，去重键稳定（含 `reservationId`），避免重复通知；
- No-Show 判定调度：每分钟扫描已结束但未签到的 `EXPECTED` 行（回扫 7 天），重复执行影响行数为 0。

提供的主要 REST API 如下表所示：

Method	Path	权限	说明
GET	<code>/api/meetings/my</code>	登录用户	我参与/组织的会议执行

			视图
GET	/api/meetings/{id}/attendees	登录用户	参与人列表
POST	/api/meetings/{id}/attendees	创建人/ADMIN	新增参与人
DELETE	/api/meetings/{id}/attendees/{uid}	创建人/ADMIN	移除参与人
POST	/api/meetings/{id}/check-in	参与人本人	签到
POST	/api/meetings/{id}/check-out	参与人本人	签退
GET	/api/meetings/{id}/attendance	相关角色	出勤总览
GET	/api/notifications	登录用户	通知列表
POST	/api/notifications/{id}/read	本人	标记已读

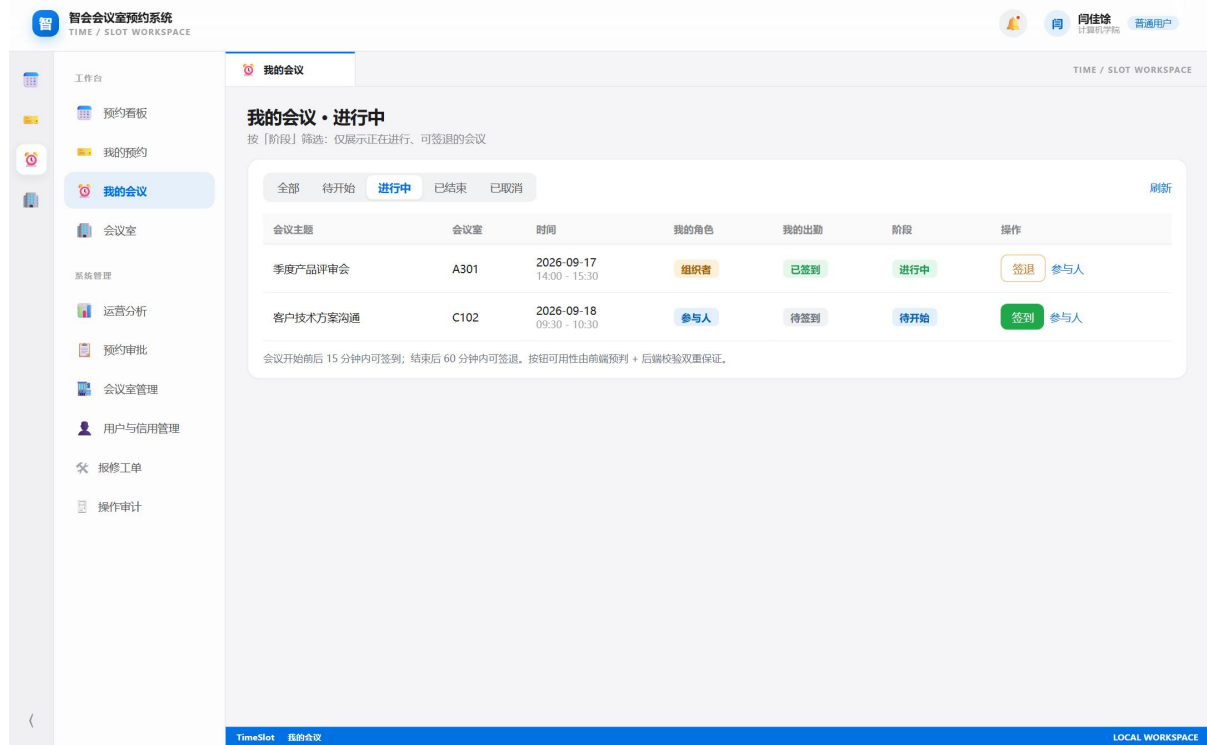


图 2 我的会议 · 进行中（支持签到/签退操作）

2.4 No-Show 缺席自动判定

会议结束后，MeetingExecutionScheduler 每分钟扫描已结束且仍处于 EXPECTED 状态的参与人行，将其置为 NO_SHOW。判定逻辑仅依赖本域 reservation_attendee 表，通过 SQL WHERE 条件限定只命中 EXPECTED 行，确保重复调度无副作用。当前版本已预留

identity 信用联动接口，待 identity 域对外 Service 就绪后接入，绝不直接修改其它域表。



图 3 No-Show 缺席记录抽屉

2.5 业务内通知与个人通知中心

NotificationService 实现了被加入/移出、开始提醒、No-Show 结果三类通知。所有通知均关联 dedup_key，调度类通知使用 INSERT IGNORE 避免重复写入。前端 NotificationBell 组件在页面顶栏 60 秒轮询未读数，点击铃铛展开通知抽屉，支持单条已读与「全部已读」。

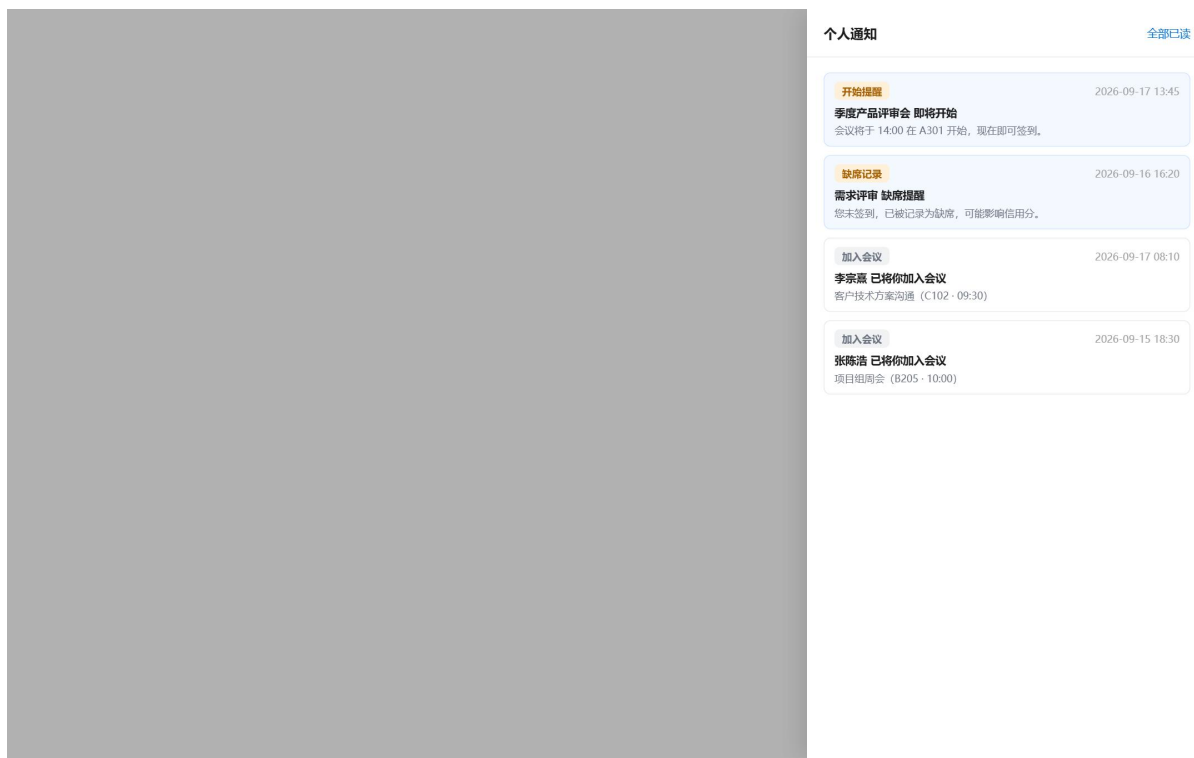


图 4 个人通知中心（开始提醒 / 缺席记录 / 加入会议）

2.6 前端闭环与个人工作台

前端代码全部位于 frontend/src/modules/meeting/，新增 MyMeetings.vue、AttendeeManager.vue、NotificationBell.vue。MyMeetings.vue 以标签页方式提供「全部 / 待开始 / 进行中 / 已结束 / 已取消」阶段筛选，在表格中展示会议主题、会议室、时间、我的角色、出勤状态与阶段，并渲染签到/签退/参与人/出勤记录操作按钮。AttendeeManager.vue 以右侧抽屉形式展示参与人列表与出勤详情，支持管理员新增参与人。MainLayout.vue 增加「我的会议」菜单与通知铃铛入口。

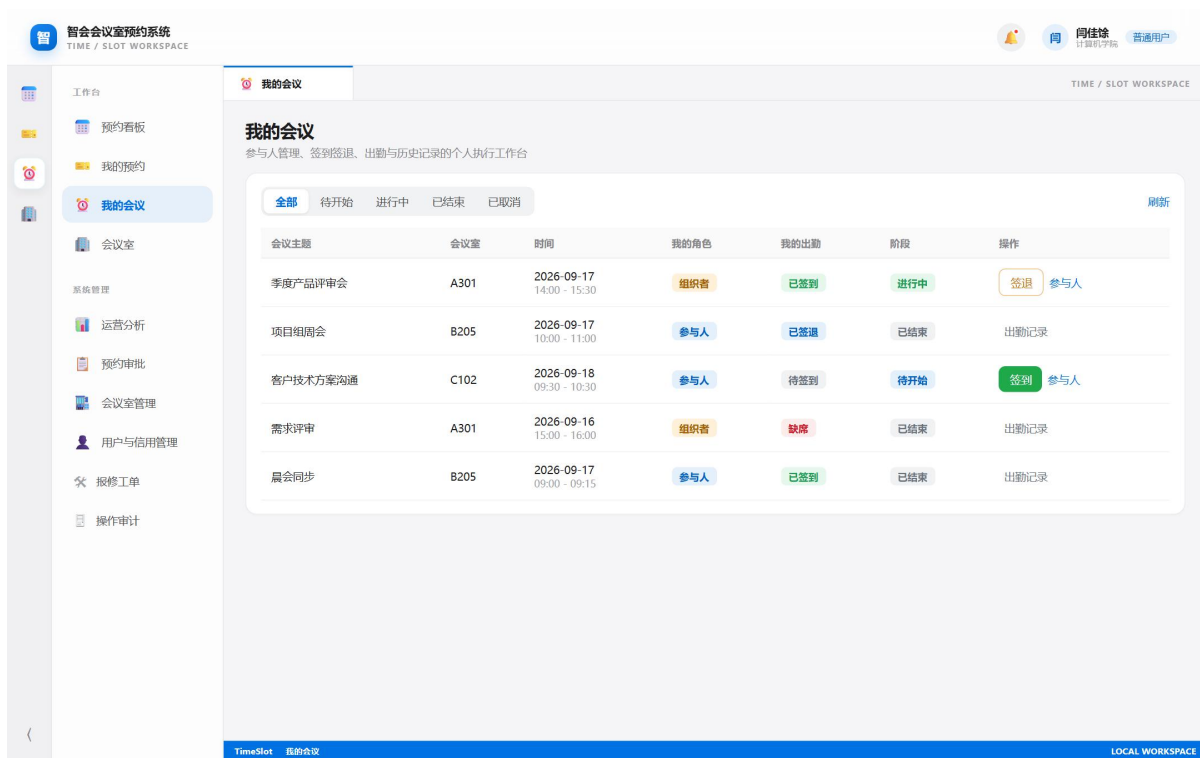


图 5 系统主框架 · 我的会议工作台

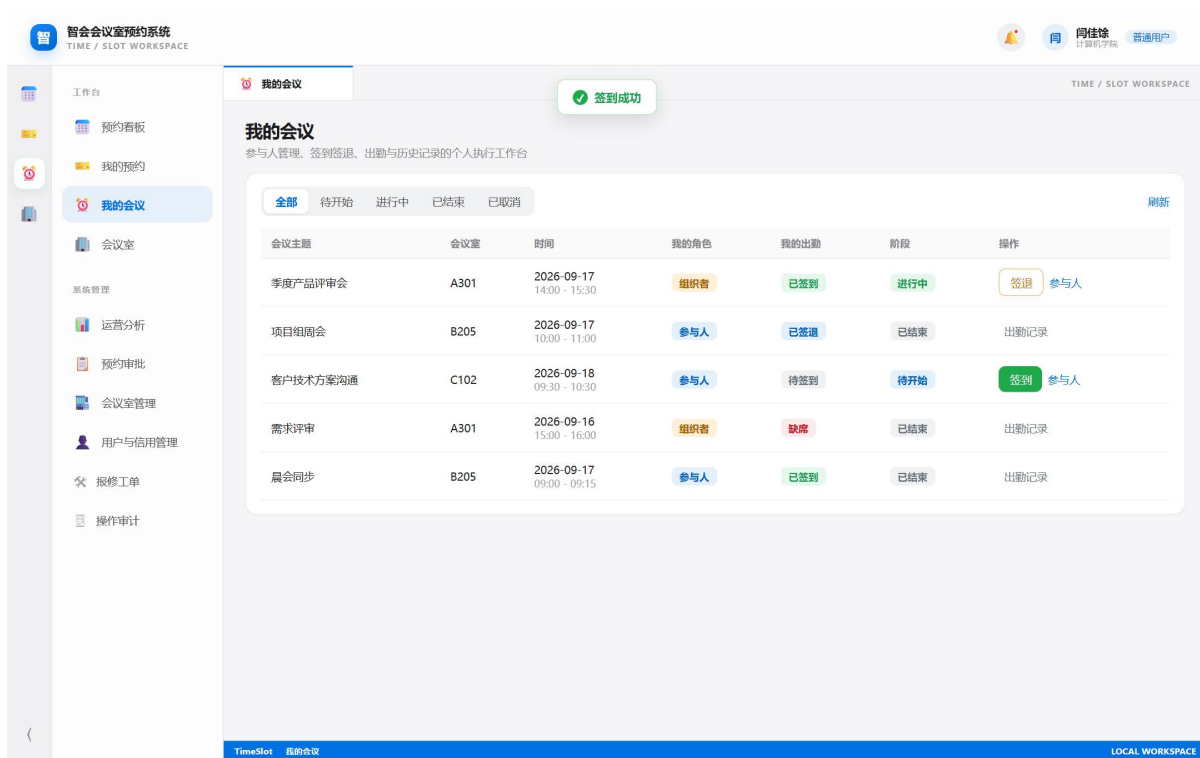


图 6 签到成功反馈 (Toast)

2.7 测试与验证

后端执行`mvn clean test`共 52 个测试全部通过，其中本轮为 meeting 域新增 37 个测试，覆盖：

- MeetingExecutionServiceTest (29)：重复参与人、用户名解析、上限、越权、CANCELLED/会后新增、组织者与出勤事实移除保护、签到窗口前后、非参与人签到、重复签到幂等、PENDING/CANCELLED 禁止签到、正常签到签退、未签到签退、重复签退幂等、宽限期外、No-Show 判定、重复判定无副作用、提醒去重键稳定、出勤可见性、预约不存在；
- ReservationQueryServiceTest (4)：只读查询过滤语义（仅 CONFIRMED、结束/开始窗口）；
- NotificationServiceTest (4)：去重键生成、未拥有/不存在已读报错、视图映射。

前端`npm run type-check`与`npm run build`均通过。

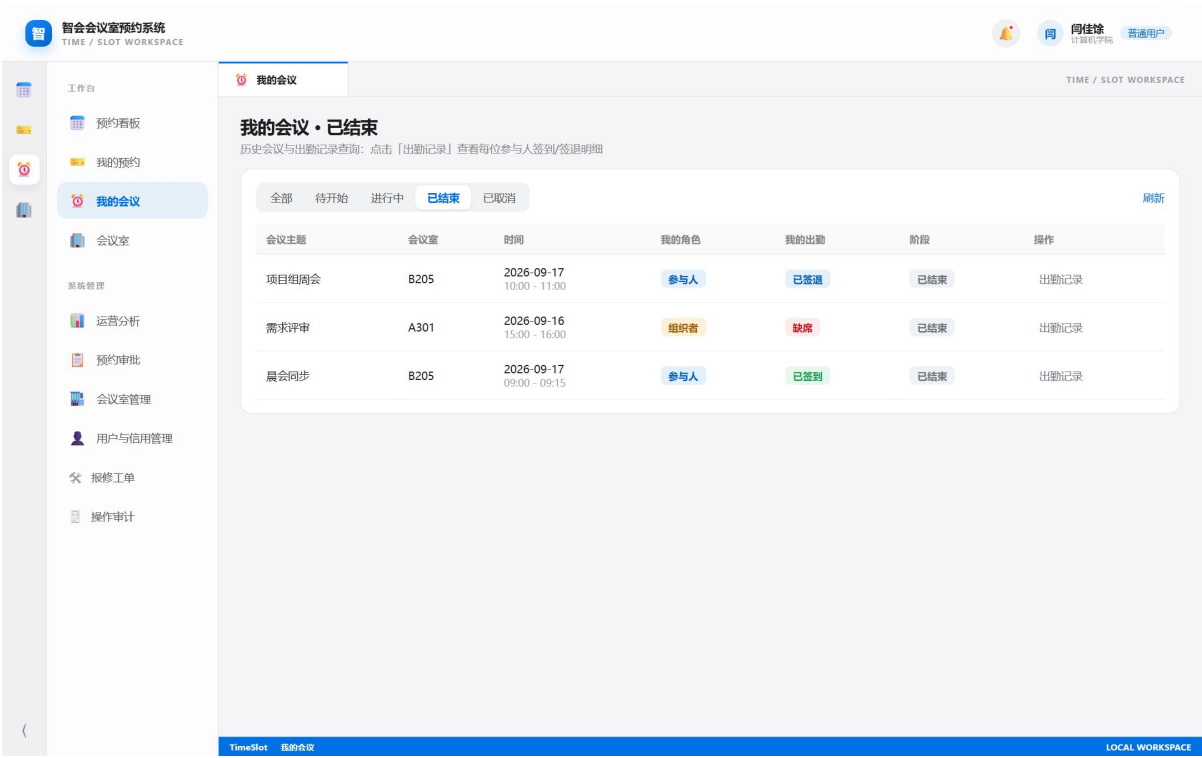


图 7 我的会议 · 已结束（出勤历史记录）

2.8 提交记录

会议执行域 feature/meeting-execution 分支共 8 个单一职责 commit，均已推送 origin：

Commit	说明
6650210	feat(meeting): 增加参与人与执行阶段数据模型
f790858	feat(reservation): 新增跨域只读预约查询服务
01244be	feat(meeting): 增加个人通知与已读能力
3c3997b	feat(meeting): 实现参与人管理、签到签退与出勤查询
0f24389	feat(meeting): 实现 No-Show 判定与开始提醒调度
71531f0	test(meeting): 覆盖签到生命周期与异常场景
cbb0693	feat(meeting): 接入个人会议执行工作台
c511ea2	feat(meeting): 添加我的会议路由与顶栏通知入口

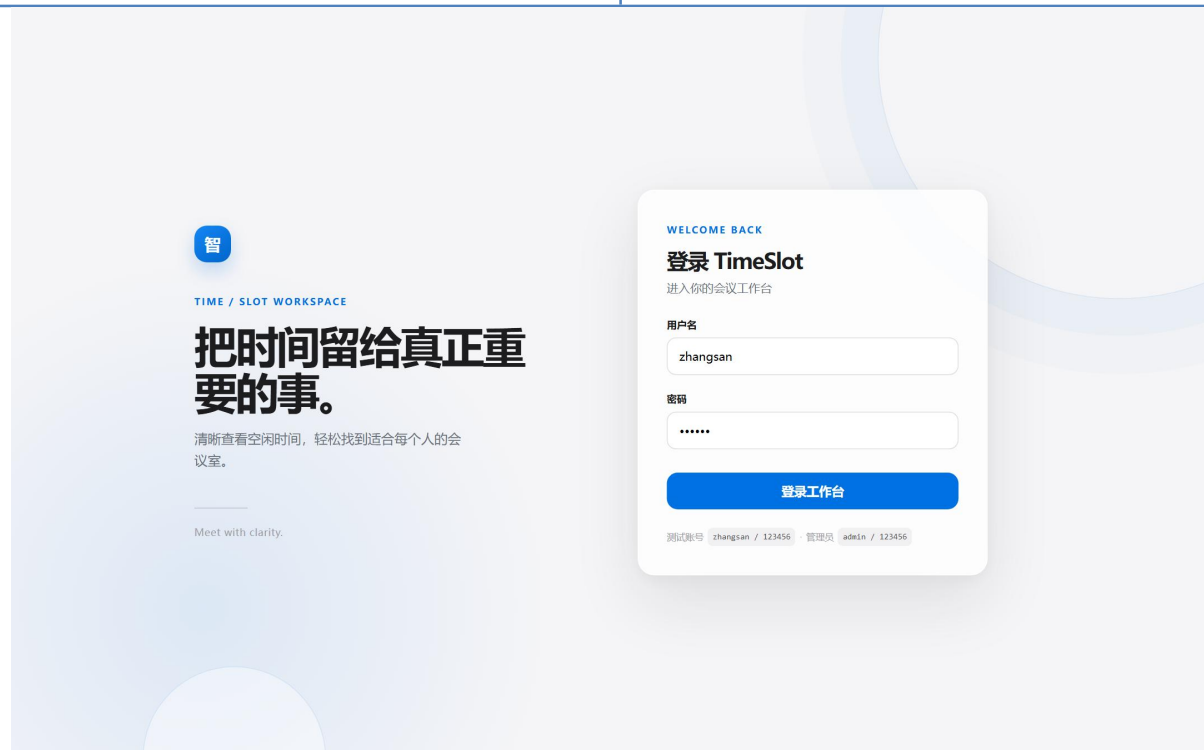


图 8 系统登录界面（用户进入 TimeSlot 工作台的入口）

三、目前存在问题，下一步的主要设计任务，具体设想与安排

经过开发、自测与团队评审，会议执行域仍存在以下待完善项：

目前存在问题	下一步任务	具体设想与安排
本机 MySQL80 服务未启动且无管理员权限启动，migration 与接口联调未能在本地复跑。	复跑 migration 并完成真实链路冒烟	在共享开发库使用 `mysql -uroot meeting_room < sql/migrations/V1_5__meeting_execution.sql` 执行迁移；用 zhangsan/lisi 账号复现「创建预约 → 添加参与人 → 签到 → 签退 → No-Show」全流程。
identity 域信用分/黑名单接口尚未就绪，No-Show 后未实现自动扣分。	接入 identity 域 SPI 回调	与张陈浩确认 UserCreditService 对外接口；No-Show 判定后异步调用扣分，失败记录补偿日志。
reservation 取消/审批结果、administration 系统公告缺少对本域的通知回调。	补齐跨域事件通知	由 reservation/administration 域提供回调钩子，meeting 域注册监听器，向受影响用户发送通知。
当前签到依赖 PC 端，缺少移动端扫码签到能力。	增加移动端/扫码签到	设计二维码签到页面：组织者生成会议二维码，参与者扫码后调用 /check-in，复用现有后端逻辑。
No-Show 与开始提醒调度在当前单实例实现下可运行，多实例部署可能出现并发重复。	增加分布式锁与监控	使用数据库悲观锁或 Redis 分布式锁保证调度单节点执行；为通知与 No-Show 增加指标埋点。

综上所述，会议执行域已完成预约后用户侧执行闭环的全部 P0 与部分 P1 能力，具备可交付的前后端功能与较完整的测试覆盖。后续将优先完成真实环境联调与跨域 SPI 集成，再逐步扩展移动端签到和分布式锁。

一 本人在该设计中具体应完成的工作（霍汶洺）

项目名称	智会会议室预约与时间冲突检查系统
负责模块	特殊会议室审批 管理员运营 系统审计与统计分析
学生姓名	霍汶洺
学号	2413041630
开发分支	feature/administration-core
报告日期	2026 年 9 月 17 日

在本课程设计中，我负责 `administration` 管理域的全栈设计与实现，具体范围包括特殊会议室预约审批、管理员操作审计、运营统计分析和后台管理页面。该模块需要在不越过 `reservation` 预约域数据所有权的前提下，实现管理员侧从待审批查询、审批决策到行为留痕和数据分析的完整链路。

序号	工作包	本人应完成的具体内容
1	审批业务闭环	实现待审批列表、预约详情、审批通过、审批驳回和强制取消；严格遵守 <code>PENDING</code> 到 <code>CONFIRMED</code> 或 <code>REJECTED</code> 的状态转换规则。
2	审计与追溯	审批成功后写入 <code>approval_record</code> 和 <code>operation_log</code> ；提供按操作人、操作类型、业务类型和时间范围查询的审计接口。
3	运营统计	基于真实预约数据计算预约总数、取消率、热门会议室、高峰时段、审批通过率

序号	工作包	本人应完成的具体内容
		及审批等待 SLA，不使用前端写死数据。
4	管理后台前端	完成预约审批、操作审计和运营分析页面，实现筛选、详情查看、审批操作、图表展示和 CSV 导出。
5	数据库与接口	新增 V1_6 migration、查询索引、DTO、Mapper、Service 与 Controller，所有 API 统一返回 ApiResponse。
6	质量与交付	完成权限、状态流转、重复审批、审计写入、查询过滤和统计逻辑测试；维护交付报告、跨域契约说明和规范化 Git 提交。

工作边界与验收标准

本人的核心边界是拥有 approval_record、operation_log 和运营统计读取模型，不直接修改 reservation 表状态。审批写操作必须调用预约域公开生命周期服务。中期验收以特殊会议室预约能够真实完成待审批到通过或驳回、关键管理员操作可追溯、统计图表来自后端真实聚合、普通用户访问管理接口返回 403 为准。

二 课程设计开始以来所做的具体工作和取得的进展

本节按照工作内容、实现结果和证明材料展开。文字说明用于交代实现思路、数据来源和验收结论，截图用于证明分支提交、真实页面、数据库联调和自动化测试均已实际完成。各项证据与任务书要求的对应关系如下。

证据	证明对象	可核验结论
图 1	分支与提交记录	证明在 feature/administration-core 开发，并按功能、测试和文档拆分提交
图 2	管理员预约审批	证明真实预约数据已接入，管理员能够查看不同状态预约并执行审批相关操作
图 3	操作审计	证明关键管理员行为已经落库，可按条件筛选并导出
图 4	运营分析	证明统计结果来自后端聚合，并已增加审批队列健康指标
图 5	自动化测试	证明后端 41 项测试和前端类型检查通过

2.1 需求梳理与分支管理

课程设计开始后，我先阅读核心业务契约、数据库基线、域边界和团队开发规范，将任务拆解为审批闭环、审计查询、运营统计、CSV 导出、前端页面、测试与文档七个子任务。随后在 feature/administration-core 分支开发，坚持 Conventional Commits，不直接在 main 分支修改。当前分支相对 main 包含 13 个阶段提交，主要成果按后端、前端、测试和文档独立提交，便于评审与回滚。

截图证明：当前工作分支为 feature/administration-core，相对 origin/main 包含 13 个阶段提交；代码差异共涉及 41 个文件，新增 2122 行、删除 13 行。提交主题覆盖后端功能、前端接入、测试、修复和文档，与任务拆分保持一致。

2.2 后端审批闭环与跨域协作

后端已完成 8 个管理员接口，包括预约列表、预约详情、审批通过、审批驳回、强制取消、审计日志查询、审计 CSV 导出和运营统计。所有接口均通过 hasRole ADMIN 进行方法级权限控制。审批 Service 使用事务编排：先调用 ReservationLifecyclePort，由薄适配器委托预约域公开的 ReservationLifecycleService 完成行锁和状态流转，随后写入审批记录与操作日志；任一步骤失败均整体回滚，避免产生状态已改变但审计缺失的不一致。

方法	接口	阶段实现
GET	/api/admin/reservations	按状态查询管理员预约列表
GET	/api/admin/reservations/{id}	预约详情及审批历史
POST	/api/admin/reservations/{id}/approve	PENDING 转为 CONFIRMED
POST	/api/admin/reservations/{id}/reject	PENDING 转为 REJECTED，原因必填
POST	/api/admin/reservations/{id}/force-cancel	管理员强制取消并记录原因
GET	/api/admin/audit-logs	组合条件查询操作日志
GET	/api/admin/audit-logs/export	导出 UTF-8 CSV
GET	/api/admin/statistics	聚合预约、审批和会议室运营指标

2.3 管理后台审批页面

前端业务代码集中在 `frontend/src/modules/administration`。预约审批页面支持状态筛选、详情抽屉、审批历史、通过、驳回、强制取消和等待时长提示。提交期间会锁定对应操作，防止管理员重复点击；对加载失败和业务错误给出明确反馈。图 2 展示了管理员查看全部预约时的真实后端数据，共可见 6 条不同状态的预约记录。

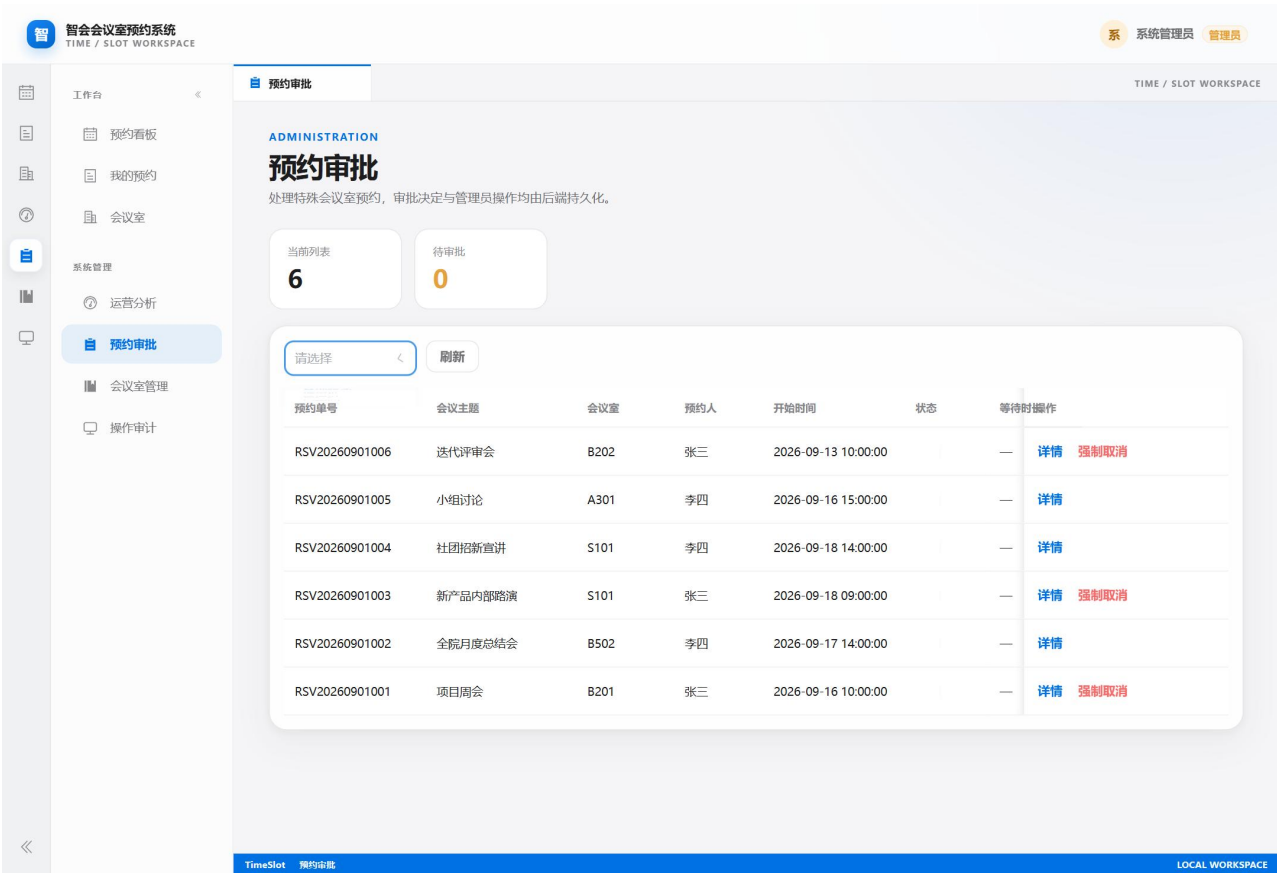


图 2 管理员预约审批页面截图

截图证明：页面从后端读取到 6 条预约记录，列表同时呈现预约编号、会议主题、会议室、预约人、开始时间和状态。管理员可以进入详情，并根据当前状态执行通过、驳回或强制取消；不满足状态条件的操作不会开放。

2.4 操作审计与安全导出

审计模块已覆盖 `APPROVE_RESERVATION`、`REJECT_RESERVATION` 和 `FORCE_CANCEL_RESERVATION`，支持操作人 ID、操作类型、业务类型和时间区间组合筛选。导出功能读取同一后端查询结果生成 CSV，并增加换行清洗、引号转义和公式注入中

和处理。同时对审计内容长度和代理 IP 长度进行保护，避免合法业务操作因数据库字段超长而失败。

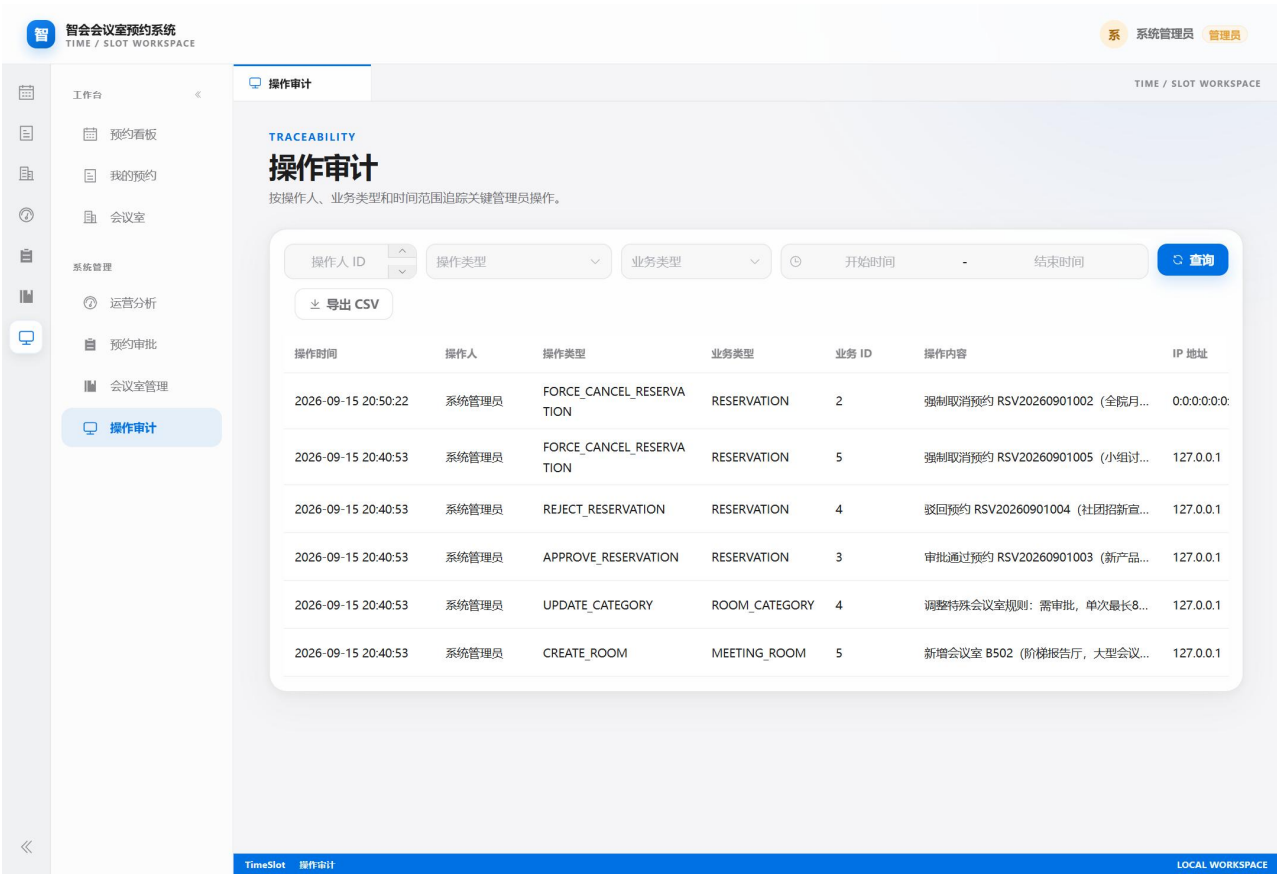


图 3 管理员操作审计与筛选导出页面截图

截图证明：审计列表已记录强制取消、驳回、通过、会议室创建和分类调整等真实操作，每条记录均包含操作时间、操作人、操作类型、业务类型、业务编号、操作内容和 IP 地址；页面提供组合筛选与 CSV 导出口。

2.5 运营分析与创意功能

运营统计直接由 MySQL 中的 reservation 和 approval_record 聚合，已实现预约总数、取消数、取消率、热门会议室使用时长、高峰时段和每日审批决策趋势。为提高管理价值，我增加了 Approval Pulse 审批队列健康雷达，实时展示当前待审批量、24 小时 SLA 超时量、平均等待、最长等待和区间审批通过率。该模块只展示真实数据；meeting 域尚未提供签到事实时，没有伪造 No-Show 指标。

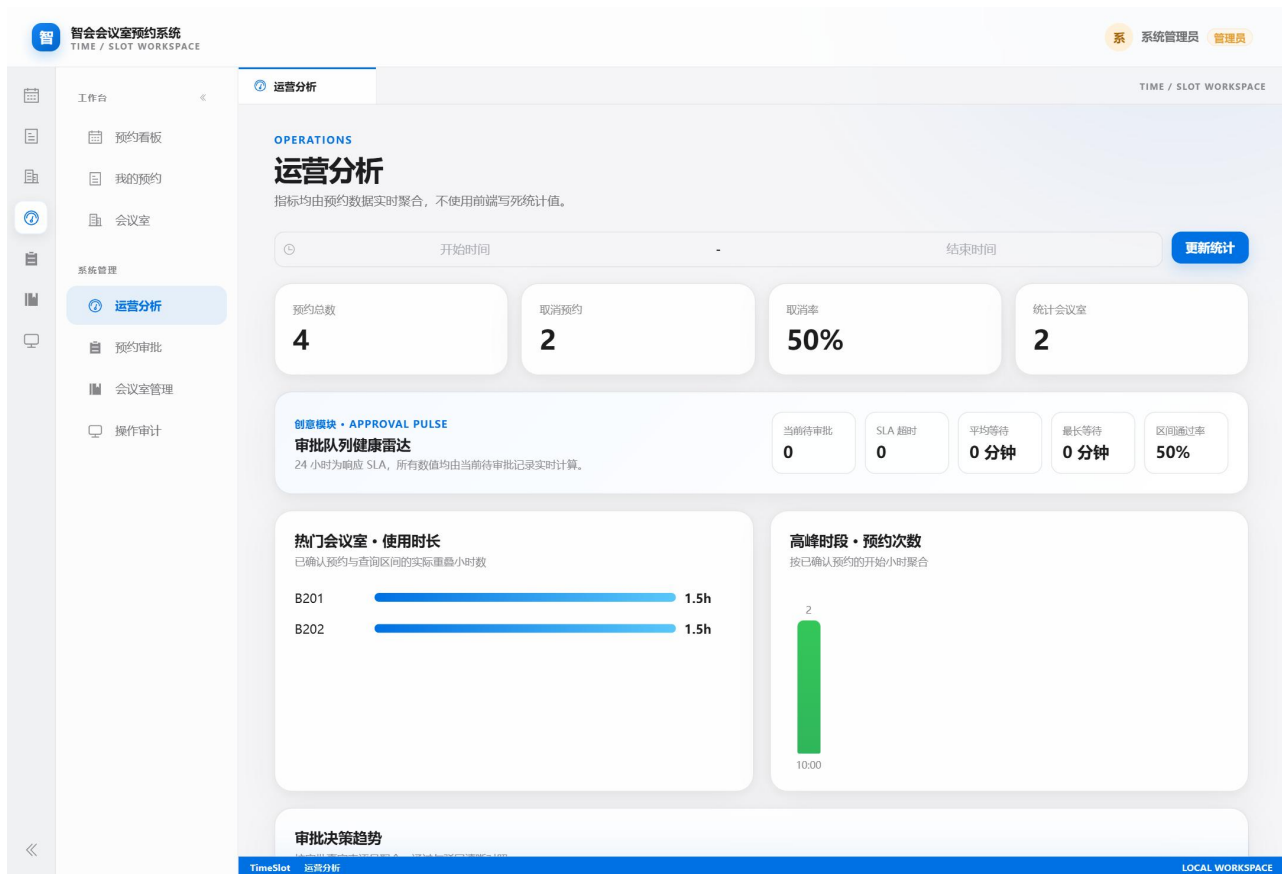


图 4 运营分析与审批队列健康雷达截图

截图证明：当前数据区间内共统计 4 条预约，其中 2 条取消，取消率为 50%；B201 与 B202 的使用时长均为 1.5 小时，高峰时段预约数为 2。Approval Pulse 同步展示待审批量、SLA 超时量和审批通过率，页面没有显示缺少真实数据来源的 No-Show 率。

2.6 数据库设计与查询优化

新增 `sql/migrations/V1_6__administration_management.sql`，在不改写历史 migration 和 schema 快照的前提下，为审批时间、审批动作、审计业务时间、审计操作人时间、预约状态区间和预约状态创建时间补充索引。本地使用 MySQL 8.0.26 完成初始化与联调，登录、会议室列表、待审批列表和运营统计接口均返回 SUCCESS。联调过程中发现普通 @Select 中误用 XML 比较运算符实体会导致真实数据库 SQL 语法错误，已修复并新增回归检查。

2.7 自动化测试与阶段质量

截至 2026 年 9 月 17 日，后端共执行 41 项自动化测试，失败 0、错误 0、跳过 0；前端 vue-tsc 类型检查通过。测试覆盖普通用户访问管理接口返回 403、审批通过、审批驳回、重复审批、强制取消、审批与操作日志写入、审计查询过滤、统计结果计算、生命周期适配和 Mapper SQL 规则。测试结果说明核心状态流转、权限边界和统计计算已具备较稳定的阶段基础。

截图证明：后端测试结果为 Tests run 41、Failures 0、Errors 0、Skipped 0，并显示 BUILD SUCCESS；前端 npm run type-check 完成且未报告类型错误。

2.8 当前完成度概述

验收项	状态	说明
待审批与审批闭环	已完成	通过、驳回、重复审批和强制取消均已实现
审批记录与操作审计	已完成	真实落库、组合查询、CSV 导出
管理员前端	已完成	审批、审计、运营三个页面已接入真实 API
运营图表	已完成	热门会议室、高峰时段、决策趋势、Approval Pulse
数据库 migration	已完成	V1_6 文件与查询索引已验证
自动化测试	已完成	41 项后端测试和前端类型检查通过
No-Show 率	待依赖	等待 meeting 域提供真实签到读取模型
主线集成	进行中	等待预约域前置提交合入 main 后整理依赖

按上述 8 个验收项统计，6 项已经完成，1 项正在进行，1 项等待跨域数据，因此当前可量化完成率为 75%。该比例只计算已完成项，不把主线集成或外部依赖计入完成，能够较真实地反映当前阶段。

三 目前存在的问题和下一步主要设计任务

3.1 目前存在的问题

问题	当前原因	可能影响	解决措施
跨分支依赖	预约生命周期服务尚需按团队顺序合入主线	合并顺序错误可能产生重复代码或共享文件冲突	先合入 reservation 前置提交，再 rebase 本分支并执行全量测试
No-Show 数据	meeting 域尚未提供签到或出席结果读取模型	现阶段无法形成可信的未到场率	保留指标限制说明，接口可用后再增加真实统计
数据库测试	MySQL 联调已完成，但事务集成测试尚未纳入自动化流程	三张业务表的一致性主要依赖人工联调证明	补充可重复执行的事务与并发审批集成测试
查询与导出	当前数据规模较小，列表尚未全面分页	数据量增长后可能出现响应变慢或导出占用过高	增加分页、稳定排序、EXPLAIN 检查和分批导出
部署配置	当前仍使用开发环境变量和本机数据库	生产环境的密钥、最小权限与日志保护尚未定型	补充配置模板、最小权限账号、日志脱敏和启动检查

3.2 下一步的主要设计任务

下一阶段将以可合并、可重复验收和可扩展为目标，优先解决跨域集成和真实数据库测试，再补充依赖 meeting 域的数据指标，最后完成性能、交互与文档收尾。具体安排如下。

计划时间	任务	具体安排	预期结果
9 月 18 日至 20 日	主线集成与差异整理	同步最新 main，确认预约生命周期提交已合入；rebase 后解决共享路由和布局冲突；重新执行全量测试。	形成可审查 PR
9 月 21 日至 23 日	MySQL 事务集成测试	构造 PENDING 预约，验证通过、驳回、重复审批、强制取消以及三张表事务一致性，并覆盖并发审批。	核心闭环可重复验证
9 月 24 日至 25 日	统计模型补充	与 meeting 域确认签到读取接口；数据可用后增加 No-Show 率和口径说明，否则保留限制说明。	统计口径真实可解释
9 月 26 日至 27 日	分页性能与体验	为预约和审计列表增加分页；使用 EXPLAIN 检查索引；完善空状态、加载状态、移动端布局和导出反馈。	提高稳定性与可用性
9 月 28 日至 30 日	最终验收与文档	完成前后端构建、migration 验证、管理员与普通用户验收；更新接口、部署、测试和最终报告。	形成最终提交材料

3.3 具体设计设想

审批一致性方面：

- 继续以 ReservationLifecyclePort 作为管理域唯一的预约状态写入口，禁止直接调用 ReservationMapper。
- 在同一事务中完成状态转换、approval_record 写入和 operation_log 写入；通过行锁保证重复审批只有一次成功。

统计扩展方面：

- 统一统计区间采用左闭右开语义，前端明确展示统计时间范围和指标计算口径。
- No-Show 仅在 meeting 域提供真实出席状态后上线，并在接口响应中明确分母、排除条件与数据更新时间。

质量保障方面：

- 增加真实 MySQL 集成测试与并发场景，继续保留 Mapper SQL 规则回归测试。
- 合并前固定执行 npm ci、npm run type-check、npm run build 和 mvn clean test，并保留可复核的测试记录。

3.4 下一阶段验收方式

验收对象	通过标准	证明材料
主线集成	功能分支可基于最新 main 完成 rebase，前后端能够启动	Git 记录、启动日志和页面检查
审批一致性	通过或驳回后状态、approval_record 和 operation_log 一致	MySQL 集成测试与数据库查询结果
权限安全	普通用户访问所有 admin 接口均返回 403	Controller 安全测试与接口实测
统计可信度	所有看板指标能够说明数据表、时间范围和计算口径	接口响应、SQL 结果和页面截图
最终质量	npm 构建、类型检查和 mvn 测试全部通过	构建记录与最终测试报告

四 中期检查结论

目前本人承担的审批审计与运营分析模块已经完成主要功能实现和本地运行验证，达到了中期阶段应形成可演示成果的要求。系统已能够展示管理员预约列表、执行审批相关操作、追溯关键行为并基于真实数据生成运营指标。按 8 个中期验收项统计，当前已有 6 项完成，可量化完成率为 75%。现存问题主要集中在团队分支集成、真实数据库自动化测试和跨域统计依赖，不属于核心页面或接口缺失。下一阶段将按照先集成、再测试、后扩展的顺序推进，确保最终成果可运行、可验证、可说明。

附录 阶段成果清单

类别	阶段成果
后端	AdministrationController、AdministrationService、AdministrationMapper、ReservationLifecycleAdapter 及相关 DTO
前端	AdministrationReservations、AuditLogs、OperationsDashboard、administration API 与类型定义
数据库	sql/migrations/V1_6__administration_management.sql
测试	权限、审批编排、强制取消、统计、生命周期适配和 Mapper SQL 共 41 项测试
文档	生命周期跨域提案、管理域交付报告和本中期检查说明报告
代码托管	Gitee feature/administration-core 分支；GitHub time-slot-audit 仓库已作为远端连接

一、本人在该设计中具体应完成的工作（张陈浩）

会议室预约与时间冲突检查系统 · 身份认证域

姓名	张陈浩
学号	2413042116
班级	软件工程 / 计算机学院
负责模块	身份认证域（identity）
指导教师	张淑娟
团队名称	尖草坪游击队

在本次课程设计中，本人负责「会议室预约与时间冲突检查系统」中的身份认证域（identity domain）。身份认证域是系统的安全与信任基座：承担登录鉴权、用户组织资料、预约资格判定与信用治理等职责，同时为其它业务域（reservation、meeting 等）提供只读的身份/资格查询能力。

1.1 功能职责

- 安全认证：基于 BCrypt 存储密码、基于 JWT 的 Stateless 鉴权；JWT 签名密钥强制从环境变量读取，禁止硬编码；
- 用户与组织管理：实现 sys_user 的增删改查、部门表 department 的维护，以及用户启用/禁用/重置密码；
- 信用与违规治理：维护信用分、违规记录 user_violation、限制期 restricted_until，支持人工调整信用与人工限制/解除；
- 自动黑名单规则：信用分低于 40 分自动进入 30 天限制期，恢复至 60 分及以上时自动解除（仅限系统自动限制）；
- 预约资格判定：对外暴露只读服务 BookingQualificationService，按「禁用 → 限制期 → 信用不足 → 正常」顺序判定；
- 数据库迁移：编写 V1_3__identity_governance.sql，新增 department、user_violation，扩展 sys_user；
- 前端管理界面：实现 UserAdmin.vue 用户与信用管理页面，覆盖用户 CRUD、信用调整、限制操作、违规记录、部门管理。

1.2 跨域边界与约束

按团队约定的五域边界，identity 域只写本域表（sys_user、department、user_violation），不直接操作 reservation、meeting、resource、administration 等

其它域的表。reservation 域在创建预约前调用

BookingQualificationService.check(userId) 进行只读资格校验；meeting 域的 No-Show 事实需通过预留 SPI 回调到 identity 域进行扣分，不能反向直接写 identity 表。

二、课程设计开始以来所做的具体工作和取得的进展

以下从安全认证、数据模型、后端服务、信用治理、前端闭环与测试验证六个方面说明具体工作，并配以系统界面截图作为完成证明。

2.1 交付范围总览

身份认证域全部能力均为真实后端实现。需求优先级与完成情况如下表所示：

优先级	能力	状态	说明
P0	安全登录与 JWT 鉴权（BCrypt + 环境变量密钥）	已完成	AuthService.login / AuthController
P0	用户 CRUD 与账号启停	已完成	UserAdminService / AuthController
P0	部门管理（增/改名，无删除）	已完成	DepartmentService / V1_3
P0	信用分调整与违规记录	已完成	UserCreditService.adjustCredit
P0	限制期（黑名单）与自动规则	已完成	UserCreditService.setRestriction + 自动阈值
P0	预约资格只读判定	已完成	BookingQualificationService.check
P1	用户与信用管理前端页面	已完成	UserAdmin.vue + stores/auth + stores/userAdmin

2.2 安全认证与登录入口

登录接口 /api/auth/login 接收 username/password，使用 BCrypt 校验密码后返回 JWT accessToken。JWT 签名密钥仅允许通过环境变量 JWT_SECRET 注入；若未配置，应

用在启动阶段直接报错，避免使用公开默认密钥带来的安全隐患。CurrentUserProvider 提供当前登录用户上下文，供所有 Controller/Service 获取 userId、role 等信息。



图 1 系统登录界面 (BCrypt + JWT 鉴权入口)

2.3 数据模型与数据库迁移

新增 V1_3__identity_governance.sql (owner: identity)，在已有 v1.2 基线上增量升级，未修改其它域表。新增 department 表保存最小组织模型；扩展 sys_user 增加 department_id、credit_score (默认 100)、restricted_until；新增 user_violation 只增事实表，记录 CREDIT_DEDUCT、CREDIT_REWARD、BLACKLIST_SET、BLACKLIST_RELEASE、ACCOUNT_DISABLE、ACCOUNT_ENABLE 六类事件，credit_change 记录分数变化，operator_id 为空表示系统自动触发。

2.4 用户与信用管理工作台

前端 UserAdmin.vue 以管理员视角展示全部用户，列表字段包含用户名、姓名、角色、部门、信用分、预约资格、限制截止与操作。预约资格由前端根据 status、restrictedUntil、creditScore 实时推导，与后端 BookingQualificationService 判定逻辑一致：账号禁用 → 限制中 → 信用不足 → 可预约。每条记录提供「信用」「限制」「记录」「编辑」「更多」操作入口。



图 2 身份工作台主框架 · 用户与信用管理

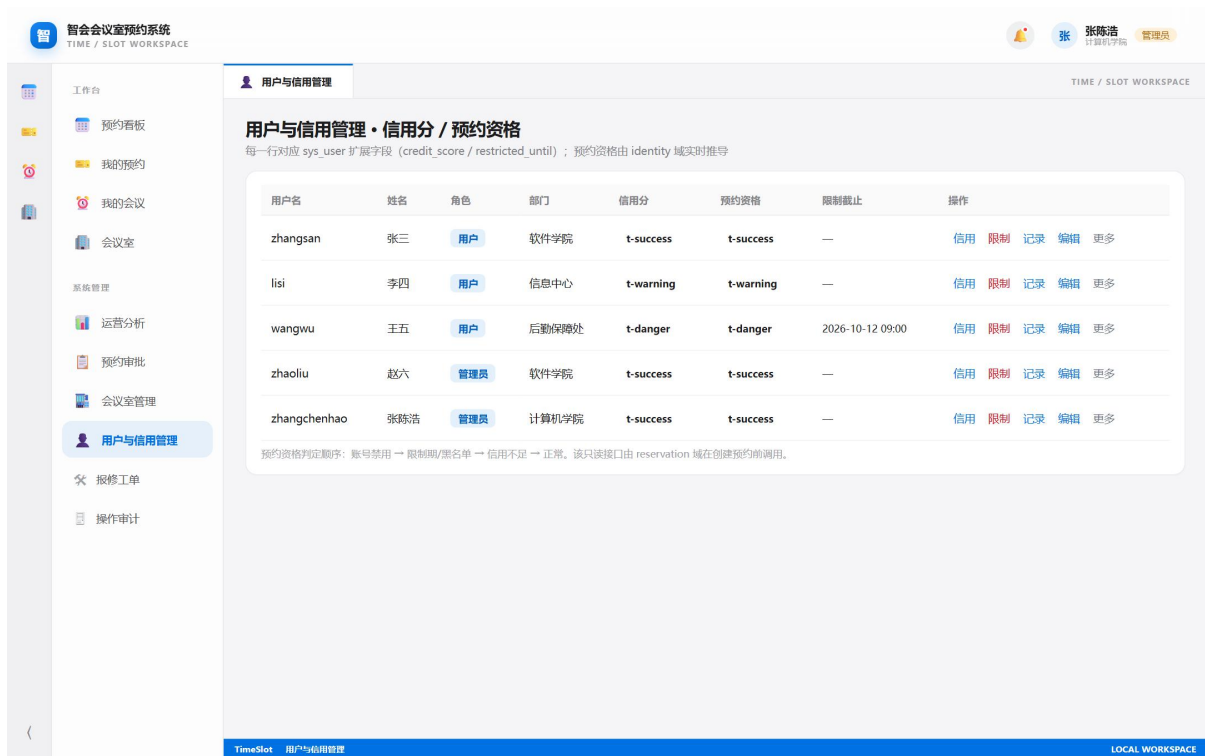


图 3 用户与信用管理列表（含信用分 / 预约资格 / 限制截止）

2.5 信用分调整与违规记录

UserCreditService.adjustCredit 要求变化量不能为 0，且必须填写调整原因；原因会写入 user_violation 表，保证每一次人工调整都有审计 trace。调整后会自动调用 applyAutoBlacklistRules：若用户原本未被限制且新分数低于 40，系统自动设置 30 天限制期并记录 BLACKLIST_SET；若当前已被系统自动限制且分数恢复到 60 及以上，系统自动解除限制并记录 BLACKLIST_RELEASE。人工设置的限制只能人工解除，不受信用分恢复影响。

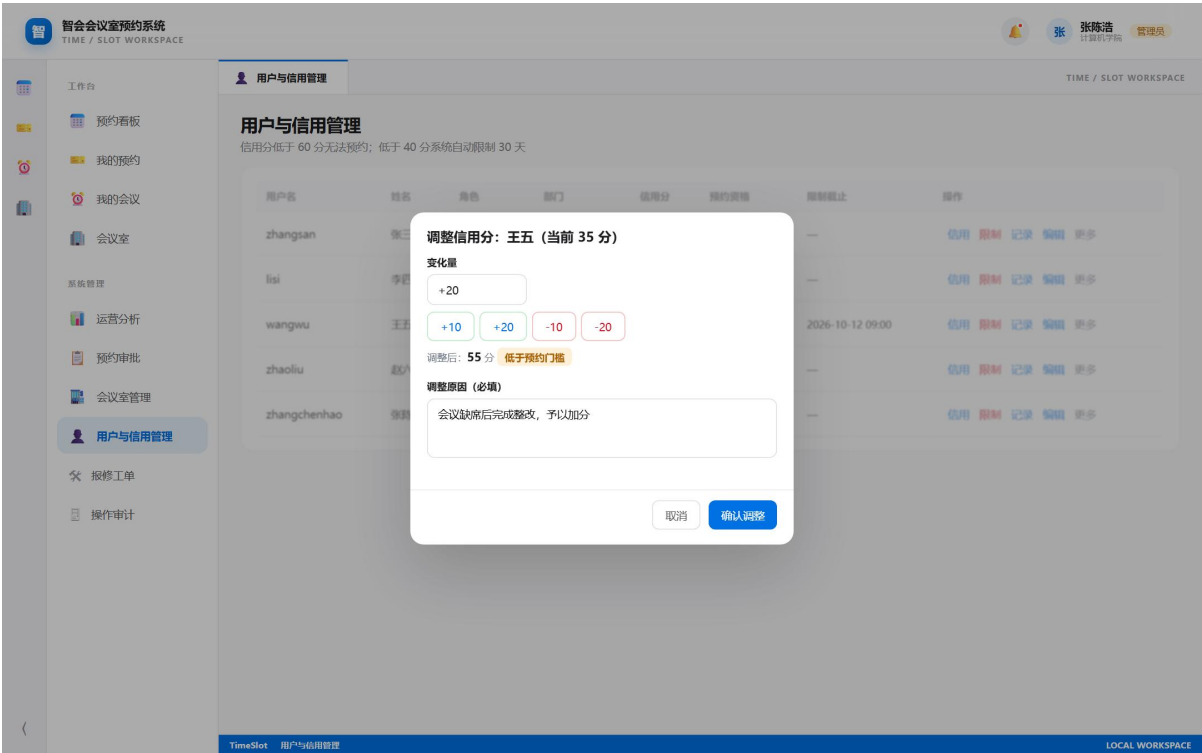


图 4 信用调整对话框（必填原因、实时预览、自动规则联动）

违规与信用记录抽屉展示用户的全部 user_violation 记录，包含类型、信用变化、原因、操作人与时间。系统自动触发的记录 operator 为空，界面显示为「系统自动」；人工调整/限制记录操作人，便于追溯。



图 5 违规与信用记录抽屉 (user_violation 只增事实表)

2.6 预约限制（黑名单）管理

setRestriction 支持「设置限制」与「解除限制」两种模式。设置限制时必须选择未来的限制截止时间并填写原因；解除限制时同样要求原因。为保护管理员自身账号，禁止对自己进行限制操作。所有限制/解除操作都会写入 user_violation，与信用调整记录共同构成完整的身份治理审计链。

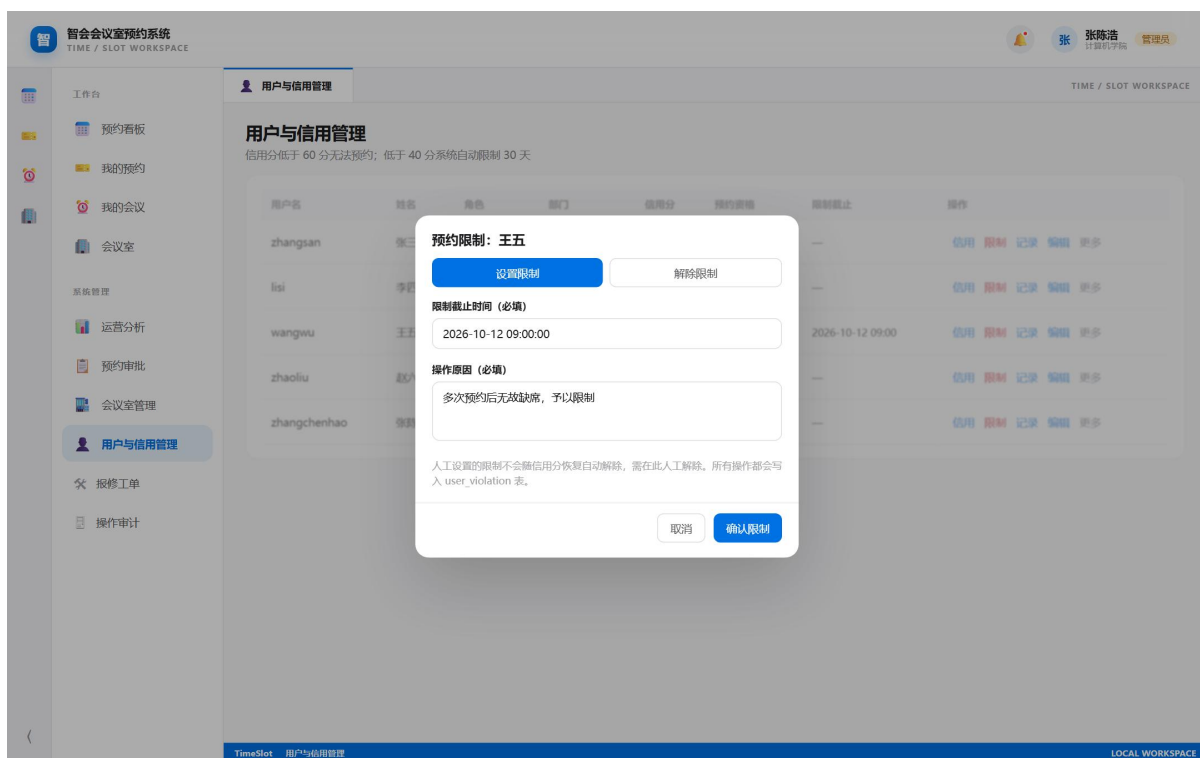


图 6 预约限制设置/解除对话框（人工限制仅人工解除）

2.7 部门管理

department 表为最小组织模型，仅支持新增、改名与说明维护，不提供删除：当部门下已挂靠用户时，删除会造成数据不一致，因此由前端禁用删除入口。部门信息在用户新增/编辑时以下拉框形式关联。

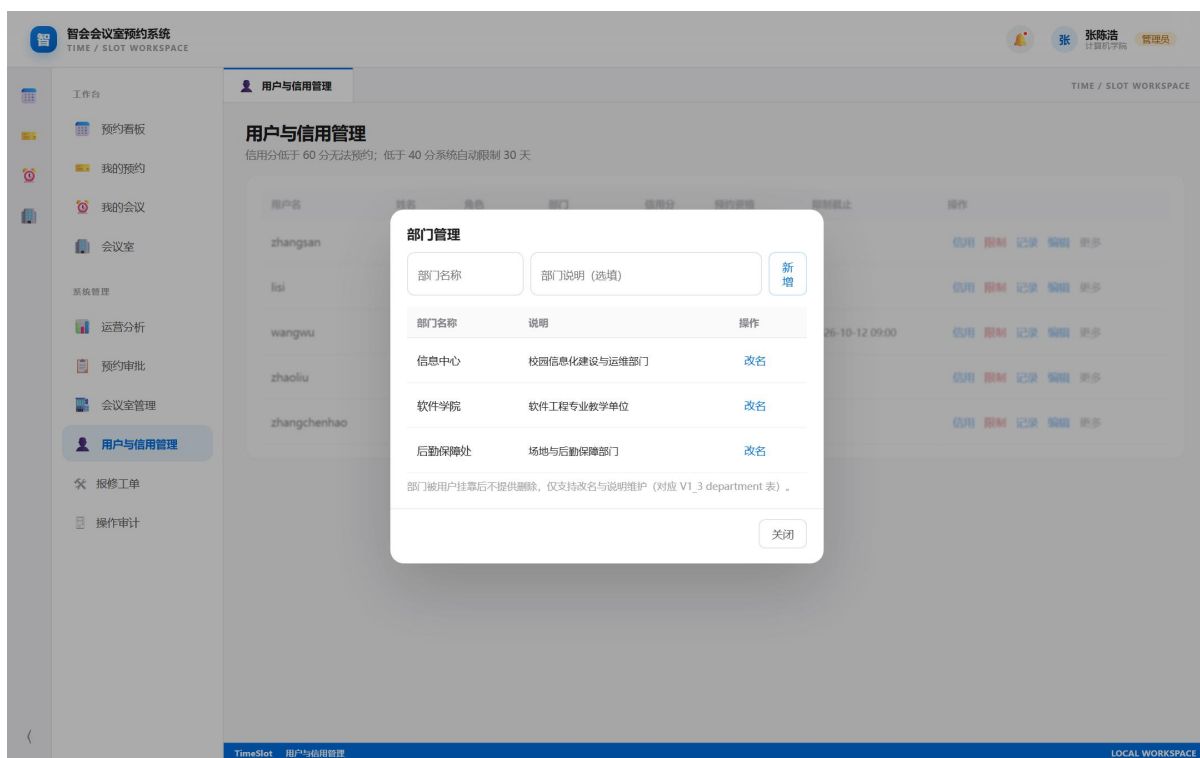


图 7 部门管理对话框（新增 / 改名 / 有用户则不删除）

2.8 对外预约资格判定服务

BookingQualificationService 是 identity 域对外暴露的只读公开能力，被 reservation 域在创建预约前调用。判定顺序为：账号禁用 → 限制期/黑名单 → 信用分不足（<60）→ 正常。返回结果包含是否允许预约、拒绝原因、当前信用分与限制截止时间。reservation 域不得直接访问 identity 的 Mapper 或数据表，必须通过该服务完成资格校验，保证域边界清晰。

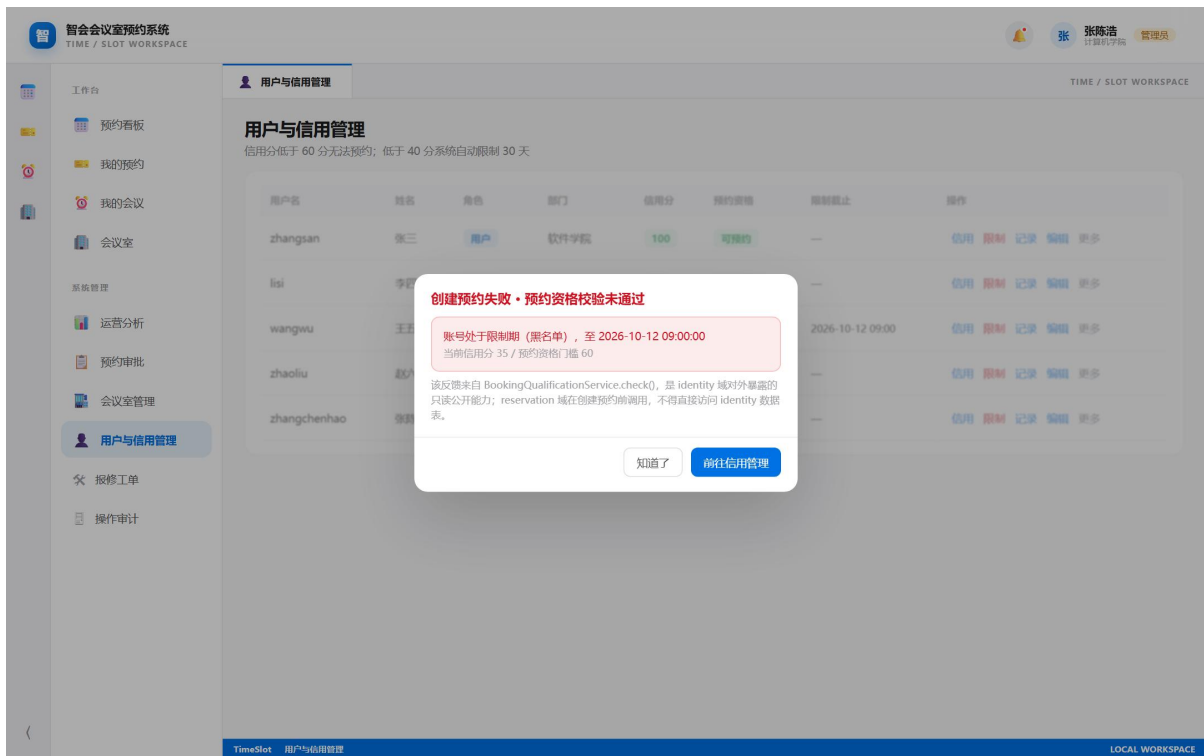


图 8 预约资格判定反馈（限制期用户无法创建预约）

2.9 关键接口与测试验证

身份认证域对外提供的关键 REST 接口如下表所示：

Method	Path	权限	说明
POST	/api/auth/login	公开	用户名密码登录，返回 JWT
GET	/api/auth/me	登录用户	获取当前登录用户信息
GET	/api/admin/users	ADMIN	查询用户列表
POST	/api/admin/users	ADMIN	新增用户
PUT	/api/admin/users/{id}	ADMIN	编辑用户资料 /部门/角色
PUT	/api/admin/users/{id}/status	ADMIN	启用/禁用账号

PUT	/api/admin/users/{id}/credit	ADMIN	调整信用分
PUT	/api/admin/users/{id}/restriction	ADMIN	设置/解除限制
GET	/api/admin/users/{id}/violations	ADMIN	查询违规/信用记录
GET	/api/admin/departments	ADMIN	部门列表

后端执行 `mvn test` 时, `UserCreditServiceTest` 共 13 个单元测试全部通过, 覆盖:

- 信用扣分/加分并记录原因 (`adjustCredit`) ;
- 变化量为 0 时校验拒绝;
- 扣分下限钳制在 0 分并记录实际变化量;
- 低于 40 分自动触发 30 天系统黑名单;
- 高于 40 分不触发自动限制;
- 系统自动限制恢复至 60 分后自动解除;
- 人工限制在信用恢复后不会被自动解除;
- 人工设置限制与解除限制并记录原因;
- 限制截止时间必须晚于当前时间;
- 禁止管理员对自己进行限制操作;
- 解除限制时若未在限制期则幂等无操作;
- 查询违规记录并正确映射操作人姓名;
- 查询不存在的用户返回 `RESOURCE_NOT_FOUND`。

前端 `npm run type-check` 与 `npm run build` 均通过。

2.10 提交记录

本人在 `identity` 域共 5 个单一职责 `commit`, 均已推送 `origin`:

Commit	说明
ffb9bcf	fix(identity): 移除公开 JWT 默认密钥
dbefb68	feat(identity): 增加部门与用户资料管理

2050a1b	feat(identity): 实现账号状态与预约资格判定
7469032	feat(identity): 实现信用调整、违规记录与黑名单治理
17322a6	feat(frontend): 实现用户与信用管理界面及身份治理接口层

三、目前存在问题，下一步的主要设计任务，具体设想与安排

经过开发、自测与团队评审，身份认证域仍存在以下待完善项：

目前存在问题	下一步任务	具体设想与安排
本机 MySQL 未启动且无管理员权限启动，V1_3 migration 与接口联调未能在本地完整复跑。	在共享开发库复跑 migration 并完成链路冒烟	使用 <code>mysql -uroot meeting_room < sql/migrations/V1_3__identity_governance.sql</code> 执行迁移；用 admin 账号验证「新增用户 → 调整信用 → 自动黑名单 → 解除限制」全流程。
meeting 域的 No-Show 缺席事实尚未自动回调到 identity 域进行信用扣分。	打通 No-Show → 信用扣分 SPI	与闫佳馥约定 SPI 接口：meeting 域在判定 No-Show 后调用 identity 提供的 <code>adjustCredit</code> 扣分入口，原因固定为「会议缺席」，并记录 <code>user_violation</code> 。
预约次数限制（P1，如单日/单周最大预约次数）尚未落地。	实现柔性预约次数门槛	在 <code>BookingQualificationService</code> 中增加近 N 天已预约次数统计，超过阈值时返回「预约次数超限」拒绝原因。
前端 UserAdmin.vue 依赖 auth / userAdmin / monitor 多个全局 store，存在循环引用与职责耦合风险。	前端 store 解耦	将 monitor 操作日志抽成独立 Hook，userAdmin 只保留数据与 CRUD 方法，页面组件只负责 UI 状态。

JWT 只有 accessToken，缺 少刷新/续期机 制，长期会话体验 不佳。	增加 JWT 刷新 能力	引入 refreshToken（可设更长过期时间）， accessToken 过期后通过 /api/auth/refresh 换 取新令牌。
---	-----------------	--

综上所述，身份认证域已完成安全认证、用户组织管理、信用治理、预约资格判定等全部 P0 能力，并提供了用户与信用管理前端页面与较完整的单元测试覆盖。后续将优先完成真实环境联调与跨域 SPI 集成，再逐步扩展预约次数限制与 JWT 刷新机制。